Chapter 5

Machine Scheduling and Job Shop Scheduling

5.1 Introduction

This chapter focuses on job shops. There are n jobs and each job visits a number of machines following a predetermined route. In some models a job may visit any given machine at most once and in other models a job may visit each machine more than once. In the latter case it is said that the job shop is subject to recirculation. A generalization of the basic job shop is a so-called flexible job shop. A flexible job shop consists of a collection of workcenters and each workcenter consists of a number of identical machines in parallel. Each job follows a predetermined route visiting a number of workcenters; when a job visits a workcenter, it may be processed on any one of the machines at that workcenter.

Job shops are prevalent in industries where each customer order is unique and has its own parameters. Wafer fabs in the semiconductor industry often function as job shops; an order usually implies a batch of a certain type of item and the batch has to go through the facility following a certain route with given processing times. Another classical example of a job shop is a hospital. The patients in a hospital are the jobs. Each patient has to follow a given route and has to be treated at a number of different stations while going through the system.

M.L. Pinedo, *Planning and Scheduling in Manufacturing and Services*,
DOI: 10.1007/978-1-4419-0910-7_5, © Springer Science + Business Media, LLC 2009

Some of the job shop problems considered in this chapter are, from a mathematical point of view, special cases of the project scheduling problem with workforce constraints described in Section 4.6. These job shop problems are NP-hard and cannot be formulated as linear programs. However, they can be formulated either as integer programs or as disjunctive programs.

This chapter is organized as follows. The second section focuses on job shops that consist of a single workcenter; these job shops are basically equivalent to either a single machine or a number of machines in parallel. Various objective functions are considered. The third section presents a mathematical programming formulation of a job shop with a single machine at each workcenter and the makespan as objective. The fourth section describes the well-known shifting bottleneck heuristic which is tailor-made for the job shop and can be adapted to the flexible job shop as well. The fifth section focuses on an entirely different approach for the job shop with the makespan objective, namely the constraint programming approach. This approach has gained a considerable popularity over the last decade. The subsequent section describes the LEKIN scheduling system which is specifically designed for job shops and flexible job shops. The discussion section lists some other popular solution techniques that have been used in practice for job shop scheduling.

All the models considered in this chapter assume that jobs are not allowed to be preempted. A job, once started on a machine, remains on that machine until it is completed.

5.2 Single Machine and Parallel Machine Models

A single machine is the simplest case of a job shop and a parallel machine environment is equivalent to a flexible job shop that consists of a single workcenter. Decomposition techniques for more complicated job shops often have to consider single machine and parallel machine subproblems within their framework.

Consider a single machine and n jobs. Job j has a processing time p_j, a release date r_j and a due date d_j. If $r_j = 0$ and $d_j = \infty$, then the processing of job j is basically unconstrained. It is clear that the makespan $C_{\max}$ in a single machine environment does not depend on the schedule. For various other objectives certain priority rules generate optimal schedules. If the objective to be minimized is the total weighted completion time, i.e., $\sum w_j C_j$, and the processing of the jobs is unconstrained, then the *Weighted Shortest Processing Time first (WSPT)* rule, which schedules the jobs in decreasing order of w_j/p_j, is optimal. If the objective is the maximum lateness $L_{\max}$ and the jobs are all released at time 0, then the *Earliest Due Date first (EDD)* rule, which schedules the jobs in increasing order of d_j, is optimal. Both the WSPT rule and the EDD rule are examples of *static* priority rules. A rule that is somewhat related to the EDD rule is the so-called *Minimum Slack first (MS)* rule which selects, when the machine becomes available at time t, the job with the least

slack; the slack of job j at time t is defined as $\max(d_j - p_j - t, 0)$. This rule does not operate in exactly the same way as the EDD rule, but will result in schedules that are somewhat similar to EDD schedules. However, the MS rule is an example of a *dynamic* priority rule, in which the priority of each job is a function of time.

Other objectives, such as the total tardiness $\sum T_j$ and the total weighted tardiness $\sum w_j T_j$, are much harder to optimize than the total weighted completion time or the maximum lateness. A heuristic for the total weighted tardiness objective is described in Appendix C. This heuristic is referred to as the *Apparent Tardiness Cost first (ATC)* rule. If the machine is freed at time t, the ATC rule selects among the remaining jobs the job with the highest value of

$$I_j(t) = \frac{w_j}{p_j} \exp\Big(- \frac{\max(d_j - p_j - t, 0)}{K\bar{p}} \Big),$$

where K is a so-called scaling parameter and $\bar{p}$ is the average of the processing times of the jobs that remain to be scheduled. The ATC priority rule is actually a weighted mixture of the WSPT and MS priority rules mentioned above. By adjusting the scaling parameter K the rule can be made to operate either more like the WSPT rule or more like the MS rule.

When the jobs in a single machine problem have different release dates r_j, then the problems tend to become significantly more complicated. One famous problem that has received a significant amount of attention is the nonpreemptive single machine scheduling problem in which the jobs have different release dates and the maximum lateness $L_{\max}$ has to be minimized. This problem is NP-Hard, which implies that, unfortunately, no efficient (polynomial time) algorithm exists for this problem. The problem can be solved either by branch-and-bound or by dynamic programming (a branch-and-bound method for this problem is described in Appendix B). It turns out that a procedure for solving this single machine problem is an important step within the well-known shifting bottleneck heuristic for general job shops.

When there are m machines in parallel the makespan $C_{\max}$ is schedule dependent. The makespan objective, which plays an important role when the loads of the various machines have to be balanced, gives rise to another priority rule, namely the *Longest Processing Time first (LPT)* rule. According to this rule, whenever one of the machines is freed, the longest job among those waiting for processing is selected to go next. The intuition behind this rule is clear: One would like to process the smaller jobs towards the end of the schedule since that makes it easier to balance the machines. If the smaller jobs go last, then the longer jobs have to be scheduled more in the beginning. The LPT rule, unfortunately, does not guarantee an optimal solution (it can be shown that this rule guarantees a solution that is within 33% of the optimal solution, see Exercise 5.3).

The SPT and the WSPT rules are important in the parallel machine setting as well. If all n jobs are available at $t = 0$, then the nonpreemptive SPT rule minimizes the total completion time $\sum C_j$; the nonpreemptive SPT rule

remains optimal even when preemptions are allowed. Unfortunately, minimizing the total weighted completion time $\sum w_j C_j$ in a parallel machine setting when all jobs are available at $t = 0$ is NP-Hard, so the WSPT rule does not minimize the $\sum w_j C_j$ in this case. However, the WSPT rule still can be used as a heuristic and it is guaranteed to generate a solution that is within 22% of optimality.

The more general parallel machine problem with the total weighted tardiness objective ($\sum w_j T_j$) is even harder. The ATC rule described above is applicable to the parallel machine setting as well; however, the quality of the solutions may at times leave something to be desired.

5.3 Job Shops and Mathematical Programming

Consider a job shop with n jobs and m machines. Each job has to be processed by a number of machines in a given order and there is no recirculation. The processing of job j on machine i is referred to as operation (i, j) and its duration is p_{ij}. The objective is to minimize the makespan $C_{\max}$.

The problem of minimizing the makespan in a job shop without recirculation can be represented by a so-called disjunctive graph. Consider a directed graph G with a set of nodes N and two sets of arcs A and B. The nodes N correspond to all of the operations (i, j) that must be performed on the n jobs. The so-called *conjunctive* (solid) arcs A represent the routes of the jobs. If arc $(i, j) \rightarrow (h, j)$ is part of A, then job j has to be processed on machine i before it is processed on machine h, i.e., operation (i, j) precedes operation (h, j). Two operations that belong to two different jobs and which have to be processed on the same machine are connected to one another by two so-called *disjunctive* (broken) arcs going in opposite directions. The disjunctive arcs B form m cliques of double arcs, one clique for each machine. (A clique is a term in graph theory that refers to a graph in which any two nodes are connected to one another; in this case each connection within a clique is a pair of disjunctive arcs.) All operations (nodes) in the same clique have to be done on the same machine. All arcs emanating from a node, conjunctive as well as disjunctive, have as length the processing time of the operation that is represented by that node. In addition there is a source U and a sink V, which are dummy nodes. The source node U has n conjunctive arcs emanating to the first operations of the n jobs and the sink node V has n conjunctive arcs coming in from all the last operations. The arcs emanating from the source have length zero, see Figure 5.1. We denote this graph by $G = (N, A, B)$.

A feasible schedule corresponds to a *selection* of one disjunctive arc from each pair such that the resulting directed graph is acyclic. This implies that each selection of arcs from within a clique must be acyclic. Such a selection determines the sequence in which the operations are to be performed on that machine. That a selection from a clique has to be acyclic can be argued as

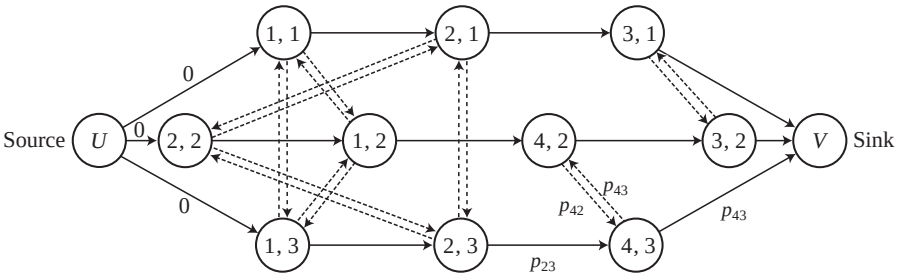

Fig. 5.1. Directed graph for job shop with the makespan as the objective

follows: If there were a cycle within a clique, a feasible sequence of the operations on the corresponding machine would not have been possible. It may not be immediately obvious why there should not be any cycle formed by conjunctive arcs and disjunctive arcs from different cliques. Such a cycle would also correspond to a situation that is infeasible. For example, let (h, j) and (i, j) denote two consecutive operations that belong to job j and let (i, k) and (h, k) denote two consecutive operations that belong to job k. If under a given schedule operation (i, j) precedes operation (i, k) on machine i and operation (h, k) precedes operation (h, j) on machine h, then the graph contains a cycle with four arcs, two conjunctive arcs and two disjunctive arcs from different cliques. Such a schedule is physically impossible. Summarizing, if D denotes the subset of the selected disjunctive arcs and the graph $G(D)$ is defined by the set of conjunctive arcs and the subset D, then D corresponds to a feasible schedule if and only if $G(D)$ contains no directed cycles.

The makespan of a feasible schedule is determined by the longest path in $G(D)$ from the source U to the sink V. This longest path consists of a set of operations of which the first starts at time 0 and the last finishes at the time of the makespan. Each operation on this path is immediately followed either by the next operation on the same machine or by the next operation of the same job on another machine. The problem of minimizing the makespan is reduced to finding a selection of disjunctive arcs that minimizes the length of the longest path (i.e., the *critical* path).

There are several mathematical programming formulations for the job shop without recirculation, including a number of integer programming formulations, see Section 4.6 and Exercise 5.4. However, the formulation most often used is the so-called disjunctive programming formulation, see Appendix A. This disjunctive programming formulation is closely related to the disjunctive graph representation of the job shop.

To present the disjunctive programming formulation, let the variable y_{ij} denote the starting time of operation (i, j). Recall that set N denotes the set of all operations (i, j), and set A the set of all routing constraints $(i, j) \rightarrow (h, j)$

which require job j to be processed on machine i before it is processed on machine h. The following mathematical program minimizes the makespan.

$$\text{minimize } C_{\max}$$

subject to

$$\begin{aligned} y_{hj} - y_{ij} &\geq p_{ij} && \text{for all } (i,j) \rightarrow (h,j) \in A \\ C_{\max} - y_{ij} &\geq p_{ij} && \text{for all } (i,j) \in N \\ y_{ij} - y_{ik} \geq p_{ik} \text{ or } y_{ik} - y_{ij} &\geq p_{ij} && \text{for all } (i,k) \text{ and } (i,j),\ i = 1,\ldots,m \\ y_{ij} &\geq 0 && \text{for all } (i,j) \in N \end{aligned}$$

In this formulation, the first set of constraints ensure that operation (h,j) cannot start before operation (i,j) is completed. The third set of constraints are called the disjunctive constraints; they ensure that some ordering exists among operations of different jobs that have to be processed on the same machine. Because of these constraints this formulation is referred to as the disjunctive programming formulation.

Example 5.3.1 (Disjunctive Programming Formulation). Consider the following example with four machines and three jobs. The route, i.e., the machine sequence, as well as the processing times are presented in the table below.

jobs	*machine sequence*	*processing times*
1	$1,2,3$	$p_{11}=10,\quad p_{21}=8,\quad p_{31}=4$
2	$2,1,4,3$	$p_{22}=8,\quad p_{12}=3,\quad p_{42}=5,\quad p_{32}=6$
3	$1,2,4$	$p_{13}=4,\quad p_{23}=7,\quad p_{43}=3$

The objective consists of the single variable $C_{\max}$. The first set of constraints consists of seven constraints: two for job 1, three for job 2 and two for job 3. For example, one of these is

$$y_{21} - y_{11} \geq 10 \ \ (= p_{11}).$$

The second set consists of ten constraints, one for each operation. An example is

$$C_{\max} - y_{11} \geq 10 \ \ (= p_{11}).$$

The set of disjunctive constraints contains eight constraints: three each for machines 1 and 2 and one each for machines 3 and 4 (there are three operations to be performed on machines 1 and 2 and two operations on machines 3 and 4). An example of a disjunctive constraint is

$$y_{11} - y_{12} \geq 3 \ \ (= p_{12}) \quad \text{or} \quad y_{12} - y_{11} \geq 10 \ \ (= p_{11}).$$

The last set includes ten nonnegativity constraints, one for each starting time.

That a scheduling problem can be formulated as a disjunctive program does not imply that there is a standard solution procedure available that will always work satisfactorily. Minimizing the makespan in a job shop is a very hard problem and solution procedures are either based on enumeration or on heuristics. To find optimal solutions branch-and-bound methods have to be used. Appendix B provides an example of a branch-and-bound procedure applied to a job shop.

5.4 Job Shops and the Shifting Bottleneck Heuristic

Since it has turned out to be very hard to solve job shop problems with large numbers of jobs to optimality, many heuristic procedures have been designed. One of the most successful procedures for minimizing the makespan in a job shop is the *Shifting Bottleneck* heuristic.

In the following overview of the Shifting Bottleneck heuristic M denotes the set of all m machines. In the description of an iteration of the heuristic it is assumed that in previous iterations a selection of disjunctive arcs has been fixed for a subset M_0 of machines. So for each one of the machines in M_0 a sequence of operations has already been determined.

An iteration results in a selection of a machine from $M - M_0$ for inclusion in set M_0. The sequence in which the operations on this machine are to be processed is also generated in this iteration. To determine which machine should be included next in M_0, an attempt is made to determine which unscheduled machine causes in one sense or another the severest disruption. To determine this, the original directed graph is modified by deleting *all* disjunctive arcs of the machines still to be scheduled (i.e., the machines in set $M - M_0$) and keeping only the relevant disjunctive arcs of the machines in set M_0 (one from every pair). Call this graph G'. Deleting all disjunctive arcs of a specific machine implies that all operations on this machine, which originally were supposed to be done on this machine one after another, now may be done in parallel (as if the machine has infinite capacity, or equivalently, each one of these operations has the machine for itself). The graph G' has one or more critical paths that determine the corresponding makespan. Call this makespan $C_{\max}(M_0)$.

Suppose that operation (i, j), $i \in \{M - M_0\}$, has to be processed in a time window of which the release date and due date are determined by the critical (longest) paths in G', i.e., the release date is equal to the longest path in G' from the source to node (i, j) and the due date is equal to $C_{\max}(M_0)$, minus the longest path from node (i, j) to the sink, plus p_{ij}. Consider each of the machines in $M - M_0$ as a separate nonpreemptive single machine problem with release dates and due dates and with the maximum lateness to be minimized. As stated in the previous section, this problem is NP-hard; however, procedures have been developed that perform reasonably well. The minimum

$L_{\max}$ of the single machine problem corresponding to machine i is denoted by $L_{\max}(i)$ and is a measure of the criticality of machine i.

After solving all these single machine problems, the machine with the *largest* maximum lateness $L_{\max}(i)$ is selected. Among the remaining machines, this machine is in a sense the most critical or the "bottleneck" and therefore the one to be included next in M_0. Assume this is machine h, call its maximum lateness $L_{\max}(h)$ and schedule it according to the optimal solution obtained for the single machine problem associated with this machine. If the disjunctive arcs that specify the sequence of operations on machine h are inserted in graph G', then the makespan of the current partial schedule increases by at least $L_{\max}(h)$, that is,

$$C_{\max}(M_0 \cup h) \geq C_{\max}(M_0) + L_{\max}(h).$$

Before starting the next iteration and determining the next machine to be scheduled, an additional step has to be done within the current iteration. In this additional step all the machines in the original set M_0 are resequenced one by one in order to see if the makespan can be reduced. That is, a machine, say machine l, is taken out of set M_0 and a graph G'' is constructed by modifying graph G' through the inclusion of the disjunctive arcs that specify the sequence of operations on machine h and the exclusion of the disjunctive arcs associated with machine l. Machine l is resequenced by solving the corresponding single machine maximum lateness problem with the release and due dates determined by the critical paths in graph G''. Resequencing each of the machines in the original set M_0 completes the iteration.

In the next iteration the entire procedure is repeated and another machine is added to the current set $M_0 \cup h$. The shifting bottleneck heuristic can be summarized as follows.

Algorithm 5.4.1 (Shifting Bottleneck Heuristic).

Step 1. *(Initial conditions)*

Set $M_0 = \emptyset$.

Graph G *is the graph with all the conjunctive arcs and no disjunctive arcs.*

Set $C_{\max}(M_0)$ *equal to the longest path in graph* G.

Step 2. *(Analysis of machines still to be scheduled)*

Do for each machine i *in set* $M - M_0$ *the following: formulate a single machine problem with all operations subject to release dates and due dates (the release date of operation* (i, j) *is determined by the longest path in graph* G *from the source to node* (i, j)*; the due date of operation* (i, j) *can be computed by considering the longest path in graph* G *from node* (i, j) *to the sink and subtracting* p_{ij}*).*

Minimize the $L_{\max}$ *in each one of these single machine subproblems.*

Let $L_{\max}(i)$ *denote the minimum* $L_{\max}$ *in the subproblem corresponding to machine* i.

Step 3. *(Bottleneck selection and sequencing)*

Let

$$L_{\max}(h) = \max_{i \in \{M - M_0\}} (L_{\max}(i))$$

Sequence machine h according to the sequence generated for it in Step 2.

Insert all the corresponding disjunctive arcs in graph G.

Insert machine h in M_0.

Step 4. *(Resequencing of all machines scheduled earlier)*

Do for each machine $l \in \{M_0 - h\}$ the following:

Delete the corresponding disjunctive arcs from G; formulate a single machine subproblem for machine l with release dates and due dates of the operations determined by longest path calculations in G.

Find the sequence that minimizes $L_{\max}(l)$ and insert the corresponding disjunctive arcs in graph G.

Step 5. *(Stopping criterion)*

If $M_0 = M$ then STOP, otherwise go to Step 2.

The structure of the shifting bottleneck heuristic shows the relationship between the bottleneck concept and the more combinatorial concepts such as critical (longest) path and maximum lateness. A critical path indicates the location and the timing of a bottleneck. The maximum lateness gives an indication of the amount by which the makespan increases if a machine is added to the set of machines already scheduled. The following example illustrates the use of the shifting bottleneck heuristic.

Example 5.4.2 (Application of Shifting Bottleneck Heuristic). Consider the instance with four machines and three jobs described in Examples 5.3.1. The routing, i.e., the machine sequences, and the processing times are given in the following table:

jobs	*machine sequence*	*processing times*
1	1,2,3	$p_{11} = 10, \quad p_{21} = 8, \quad p_{31} = 4$
2	2,1,4,3	$p_{22} = 8, \quad p_{12} = 3, \quad p_{42} = 5, \quad p_{32} = 6$
3	1,2,4	$p_{13} = 4, \quad p_{23} = 7, \quad p_{43} = 3$

Iteration 1: Initially, set M_0 is empty and graph G' contains only conjunctive arcs and no disjunctive arcs. The critical path and the makespan $C_{\max}(\emptyset)$ can be determined easily: this makespan is equal to the maximum total processing time required for any job. The maximum of 22 is achieved in this case by both job 1 and job 2. To determine which machine to schedule first, each machine is considered as a nonpreemptive single machine maximum lateness

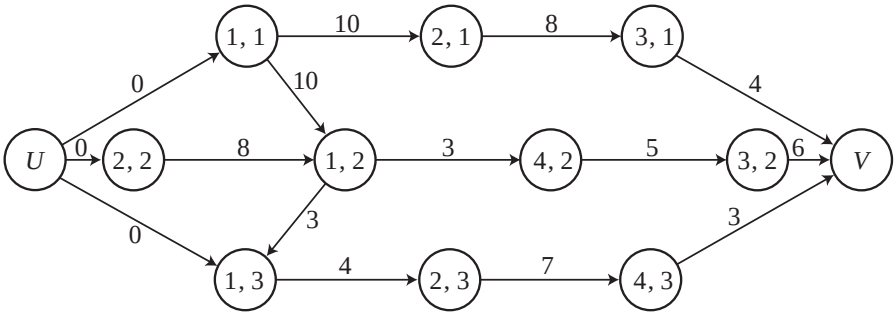

Fig. 5.2. Iteration 1 of shifting bottleneck heuristic (Example 5.4.2)

problem with the release dates and due dates determined by the longest paths in G' (assuming a makespan of 22).

The data for the nonpreemptive single machine maximum lateness problem corresponding to machine 1 is determined as follows.

jobs	1	2	3
p_{1j}	10	3	4
r_{1j}	0	8	0
d_{1j}	10	11	12

The optimal sequence turns out to be $1, 2, 3$ with $L_{\max}(1) = 5$.

The data for the subproblem associated with machine 2 are:

jobs	1	2	3
p_{2j}	8	8	7
r_{2j}	10	0	4
d_{2j}	18	8	19

The optimal sequence for this problem is $2, 3, 1$ with $L_{\max}(2) = 5$. Similarly, it can be shown that

$$L_{\max}(3) = 4$$

and

$$L_{\max}(4) = 0.$$

From this it follows that either machine 1 or machine 2 may be considered a bottleneck. Breaking the tie arbitrarily, machine 1 is selected to be included in M_0. The graph G'' is obtained by fixing the disjunctive arcs corresponding to the sequence of the jobs on machine 1 (see Figure 5.2). It is clear that

$$C_{\max}(\{1\}) = C_{\max}(\emptyset) + L_{\max}(1) = 22 + 5 = 27.$$

Iteration 2: Given that the makespan corresponding to G'' is 27, the critical paths in the graph can be determined. The three remaining machines have to be analyzed separately as nonpreemptive single machine problems. The data for the problem concerning machine 2 are:

jobs	1	2	3
p_{2j}	8	8	7
r_{2j}	10	0	17
d_{2j}	23	10	24

The optimal schedule is $2, 1, 3$ and the resulting $L_{\max}(2) = 1$. The data for the problem corresponding to machine 3 are:

jobs	1	2
p_{3j}	4	6
r_{3j}	18	18
d_{3j}	27	27

Both sequences are optimal and $L_{\max}(3) = 1$. Machine 4 can be analyzed in the same way and the resulting $L_{\max}(4) = 0$. Again, there is a tie and machine 2 is selected to be included in M_0. So $M_0 = \{1, 2\}$ and

$$C_{\max}(\{1, 2\}) = C_{\max}(\{1\}) + L_{\max}(2) = 27 + 1 = 28.$$

The disjunctive arcs corresponding to the job sequence on machine 2 are added to G'' and graph G''' is obtained. At this point, still as a part of iteration 2, an attempt may be made to decrease $C_{\max}(\{1, 2\})$ by resequencing machine 1. It can be checked that resequencing machine 1 does not give any improvement.

Iteration 3: The critical path in G''' can be determined and machines 3 and 4 remain to be analyzed. These two problems turn out to be very simple with both having a zero maximum lateness. Neither of the machines constitutes a bottleneck in any way.

The final schedule is determined by the following machine sequences: the job sequence $1, 2, 3$ on machine 1; the job sequence $2, 1, 3$ on machine 2; the job sequence $2, 1$ on machine 3 and the job sequence $2, 3$ on machine 4. The makespan is 28.

The single machine maximum lateness problem that has to be solved repeatedly within each iteration of the shifting bottleneck heuristic may actually be slightly different and more complicated than the single machine maximum lateness problem described in Section 5.2 and in Example B.4.1. In the single machine subproblem that has to be solved in Step 2 of the shifting bottleneck heuristic, the operations on the given machine may be subject to a special type of precedence constraints. It may be that an operation on the machine to be scheduled *must* be processed after certain other operations have been

completed on that machine; these precedence constraints must be satisfied because of the sequences of operations on the machines that already have been scheduled in previous iterations. Moreover, it may even be the case that in between the processing of two operations subject to these precedence constraints a certain minimum amount of time (i.e., a delay) has to elapse. The lengths of the delays are also determined by the sequences of the operations on the machines already scheduled in previous iterations. These precedence constraints are therefore referred to as *delayed* precedence constraints.

It is easy to construct examples that show the need for delayed precedence constraints in the single machine subproblem. Without these constraints the shifting bottleneck heuristic may end up in a situation with a cycle in the disjunctive graph and the corresponding schedule being infeasible.

Extensive numerical research has shown that the Shifting Bottleneck heuristic is extremely effective. When applied to a famous test problem with 10 machines and 10 jobs that had remained unsolved for more than 20 years, the heuristic found a very good solution very fast. This solution actually turned out to be optimal after a branch-and-bound procedure found the same result and verified its optimality. The branch-and-bound approach, in contrast to the heuristic, needed many hours of CPU time. The disadvantage of the heuristic is that there is never a guarantee that the solution generated is optimal.

The Shifting Bottleneck heuristic can be extended to models that are more general than the basic job shop considered above. It can be applied to more general machine environments and processing characteristics as well as to other objective functions.

The disjunctive graph formulation for the job shop problem without recirculation also extends to the job shop problem with recirculation. The set of disjunctive arcs for a machine that is subject to recirculation is now not a clique. If two operations of the same job have to be performed on the same machine a precedence relationship is given. These two operations are not connected by a pair of disjunctive arcs, since they are already connected by conjunctive arcs. The Shifting Bottleneck heuristic described can still be implemented. The jobs in the single machine subproblem are now subject to precedence constraints, which are imposed by the fact that different operations of the same job may require processing on the same machine.

The most general machine environment that can be dealt with is the flexible job shop. Each workcenter consists of a set of machines in parallel. The disjunctive graph representation can be extended to this machine setting as follows: again, all pairs of operations that have to be performed at a workcenter are linked by pairs of disjunctive arcs. If two operations are scheduled for processing on the same machine, then the appropriate disjunctive arc is selected. If two operations are assigned to different machines in the workcenter, then both disjunctive arcs are deleted from the graph, since the two operations do not affect each other's processing. The subproblem that then has to be solved in Step 2 of the Shifting Bottleneck Heuristic (wich was in the original version of the Shifting Bottleneck heuristic a single machine max-

imum lateness problem with jobs having different release dates) now becomes a parallel machine maximum lateness problem with jobs that have different release dates.

There are variations of the shifting bottleneck heuristic that can be applied to job shop problems with the total weighted tardiness objective. The disjunctive graph formulation for the job shop with the total weighted tardiness objective is slightly different from the disjunctive graph formulation for the makespan objective. Also, the objective function of the subproblem is a more complicated objective than the $L_{\max}$ objective in the subproblem described above.

The technique described above can also be adapted to flexible job shops with multiple objectives. We have already seen that the makespan objective in the original job shop problem leads to a maximum lateness objective in its single machine subproblems. The total weighted tardiness in the original job shop problem leads to a more complicated due date related objective function in the single machine subproblems. Multiple objectives in the original problem lead to multiple objectives in the single machine (or parallel machines) subproblems; the weights of the various objectives in the original problem have, of course, an impact on the weights of the objectives in the subproblems.

The heuristic approaches described above can be linked to local search procedures as described in Appendix C. For example, the shifting bottleneck heuristic can be used first to obtain a schedule with a reasonably good makespan and this solution is then fed into a local search procedure that may yield an even better solution.

5.5 Job Shops and Constraint Programming

Constraint programming is a technique that originated in the Artificial Intelligence (AI) community. In recent years, it has often been implemented in conjunction with Operations Research (OR) techniques in order to improve its overall effectiveness (see Appendix D).

Constraint programming, in its original design, only tries to find a good solution that is feasible and satisfies all the given constraints. The solutions found may, therefore, not necessarily be optimal. The constraints may include release dates and due dates for the jobs. It is possible to embed such a technique in a framework that is designed to actually minimize any due date related objective function.

Constraint programming can be applied to the basic job shop with the makespan objective as follows. Suppose that in a job shop a schedule has to be found with a makespan $C_{\max}$ that is less than or equal to a given deadline $\bar{d}$. A constraint satisfaction algorithm has to produce for each machine a sequence of the operations in such a way that the overall schedule has a makespan that is less than or equal to $\bar{d}$.

Before the actual procedure starts, an initialization step has to be done. For each operation a computation is done to determine its earliest possible starting time and latest possible completion time on the machine in question. After all the time windows have been computed, the time windows of all the operations on each machine are compared with one another. When the time windows of two operations on any given machine do not overlap, a precedence relationship between the two operations can be imposed; in any feasible schedule the operation with the earlier time window must precede the operation with the later time window. Actually, a precedence relationship may even be inferred when the time windows do overlap. Let S'_{ij} (S''_{ij}) denote the earliest (latest) possible starting time of operation (i, j) and C'_{ij} (C''_{ij}) the earliest (latest) possible completion time of operation (i, j) under the current set of precedence constraints. Note that the earliest possible starting time of operation (i, j), i.e., S'_{ij}, may be regarded as a local release date of the operation and may be denoted by r_{ij}, whereas the latest possible completion time, i.e., C''_{ij}, may be considered a local due date denoted by d_{ij}. Define the slack between the processing of operations (i, j) and (i, k) on machine i as

$$\begin{aligned}\sigma_{(i,j)\to(i,k)} &= S''_{ik} - C'_{ij} \\ &= C''_{ik} - S'_{ij} - p_{ij} - p_{ik} \\ &= d_{ik} - r_{ij} - p_{ij} - p_{ik}.\end{aligned}$$

If

$$\sigma_{(i,j)\to(i,k)} < 0$$

then, under the current set of precedence constraints, no feasible schedule exists in which operation (i, j) precedes operation (i, k) on machine i. So a precedence relationship can be imposed that requires operation (i, k) to appear before operation (i, j). In the initialization step of the procedure all pairs of time windows are compared with one another and all implied precedence relationships are inserted in the disjunctive graph. Because of these additional precedence constraints the time windows of each one of the operations can be adjusted (narrowed), i.e., this involves a recomputation of the release date and the due date of each operation.

Constraint satisfaction techniques often rely on constraint propagation. A constraint satisfaction technique typically attempts, in each step, to insert new precedence constraints (disjunctive arcs) that are implied by the precedence constraints inserted before. With the new precedence constraints in place the technique recomputes the time windows of all operations. For each pair of operations that have to be processed on the same machine it has to be verified which one of the following four cases holds:

Case 1:
If $\sigma_{(i,j)\to(i,k)} \geq 0$ and $\sigma_{(i,k)\to(i,j)} < 0$,
then the precedence constraint $(i, j) \to (i, k)$ has to be imposed.

Case 2:
If $\sigma_{(i,k)\rightarrow(i,j)} \geq 0$ and $\sigma_{(i,j)\rightarrow(i,k)} < 0$,
then the precedence constraint $(i,k) \rightarrow (i,j)$ has to be imposed.

Case 3:
If $\sigma_{(i,j)\rightarrow(i,k)} < 0$ and $\sigma_{(i,k)\rightarrow(i,j)} < 0$,
then there is no schedule that satisfies the precedence constraints already in place.

Case 4:
If $\sigma_{(i,j)\rightarrow(i,k)} \geq 0$ and $\sigma_{(i,k)\rightarrow(i,j)} \geq 0$,
then either ordering between the two operations is still possible.

In one step of the algorithm that is described later on in this section, a pair of operations has to be considered that satisfy the conditions of Case 4, i.e., either ordering between the operations is still possible. It may be the case that in this step of the algorithm many pairs of operations still satisfy Case 4. If there is more than one pair of operations that satisfy the conditions of Case 4, then a selection heuristic has to be applied. The selection of a pair is based on the sequencing flexibility this pair still provides. The pair with the lowest flexibility is selected. The reasoning behind this approach is straightforward: if a pair with low flexibility is not scheduled early on in the process, then later on in the process that pair may not be schedulable at all. So it makes sense to give priority to those pairs with a low flexibility and postpone pairs with a high flexibility. Clearly, the flexibility depends on the amounts of slack in the two orderings. One simple estimate of the sequencing flexibility of a pair of operations, $\phi((i,j)(i,k))$, is the minimum of the two slacks, i.e.,

$$\phi((i,j)(i,k)) = \min\left(\sigma_{(i,j)\rightarrow(i,k)}, \sigma_{(i,k)\rightarrow(i,j)}\right).$$

However, relying on this minimum may lead to problems. For example, suppose one pair of operations has slack values 3 and 100, whereas another pair has slack values 4 and 4. In this case, there may be only limited possibilities for scheduling the second pair and postponing a decision with regard to the second pair may well eliminate them. A feasible ordering of the first pair may not really be in jeopardy. Instead of using $\phi((i,j)(i,k))$ the following measure of sequencing flexibility has proven to be more effective:

$$\phi'((i,j)(i,k)) = \sqrt{\min(\sigma_{(i,j)\rightarrow(i,k)},\ \sigma_{(i,k)\rightarrow(i,j)}) \times \max(\sigma_{(i,j)\rightarrow(i,k)},\ \sigma_{(i,k)\rightarrow(i,j)})}.$$

So if the max is large, then the flexibility of a pair of operations increases and the urgency to order the pair goes down. After the pair of operations with the least flexibility $\phi'((i,j)(i,k))$ has been selected, the precedence constraint that retains the most flexibility is imposed, i.e., if

$$\sigma_{(i,j)\rightarrow(i,k)} \geq \sigma_{(i,k)\rightarrow(i,j)}$$

operation (i, j) must precede operation (i, k).

In one of the steps of the algorithm it also can happen that a pair of operations satisfies Case 3. When this is the case the partial schedule that is under construction cannot be completed and the algorithm has to backtrack. Backtracking can take any one of several forms. Backtracking may imply that either (i) one or more of the ordering decisions made in earlier iterations has to be annulled, or (ii) there does not exist a feasible solution for the problem in the way it was formulated and one or more of the original constraints in the problem have to be relaxed.

The constraint satisfaction procedure can be summarized as follows.

Algorithm 5.5.1 (Constraint Satisfaction Procedure).

Step 1.

Compute for each unordered pair of operations the slacks $\sigma_{(i,j)\rightarrow(i,k)}$ *and* $\sigma_{(i,k)\rightarrow(i,j)}$.

Step 2.

Check dominance conditions and classify remaining ordering decisions.

If any ordering decision is of Case 3, then BACKTRACK.

If any ordering decision is either of Case 1 or Case 2 go to Step 3; otherwise go to Step 4.

Step 3.

Insert new precedence constraint and go to Step 1.

Step 4.

If no ordering decision is of Case 4, then solution is found. STOP.

Otherwise go to Step 5.

Step 5.

Compute $\phi'((i, j)(i, k))$ *for each pair of operations not yet ordered.*

Select the pair with the minimum $\phi'((i, j)(i, k))$.

If $\sigma_{(i,j)\rightarrow(i,k)} \geq \sigma_{(i,k)\rightarrow(i,j)}$, *then operation* (i, k) *must follow* (i, j)*; otherwise operation* (i, j) *must follow operation* (i, k).

Go to Step 3.

In order to apply the constraint satisfaction procedure to a job shop problem with the makespan objective, it has to be embedded in the following framework. First, an upper bound d_u and a lower bound d_l have to be found for the makespan.

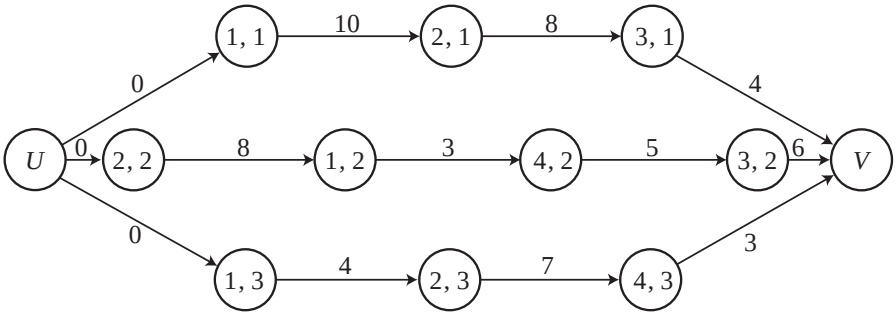

Fig. 5.3. Disjunctive graph without disjunctive arcs

Algorithm 5.5.2 (Framework for Constraint Programming).
Step 1.

Set $d = (d_l + d_u)/2$.

Apply Algorithm 5.5.1.

Step 2.

If $C_{\max} < d$, *set* $d_u = d$.

If $C_{\max} > d$, *set* $d_l = d$.

Step 3.

If $d_u - d_l > 1$ *return to Step 1.*

Otherwise STOP.

The following example illustrates the use of the contraint satisfaction technique.

Example 5.5.3 (Application of Constraint Programming to a Job Shop). Consider the instance of the job shop problem described in Example 5.3.1.

jobs	*machine sequence*	*processing times*
1	$1, 2, 3$	$p_{11} = 10, \quad p_{21} = 8, \quad p_{31} = 4$
2	$2, 1, 4, 3$	$p_{22} = 8, \quad p_{12} = 3, \quad p_{42} = 5, \quad p_{32} = 6$
3	$1, 2, 4$	$p_{13} = 4, \quad p_{23} = 7, \quad p_{43} = 3$

Consider a due date $d = 32$ when all jobs have to be completed. Consider again the disjunctive graph but disregard all disjunctive arcs, see Figure 5.3. By doing all longest path computations, the local release dates and local due dates for all operations can be established.

operations	r_{ij}	d_{ij}
(1,1)	0	20
(2,1)	10	28
(3,1)	18	32
(2,2)	0	18
(1,2)	8	21
(4,2)	11	26
(3,2)	16	32
(1,3)	0	22
(2,3)	4	29
(4,3)	11	32

Given these time windows for all the operations, it has to be verified whether these constraints already imply any additional precedence constraints. Consider, for example, the pair of operations (2,2) and (2,3) which both have to go on machine 2. Computing the slack yields

$$\begin{aligned}\sigma_{(2,3)\rightarrow(2,2)} &= d_{22} - r_{23} - p_{22} - p_{23} \\ &= 18 - 4 - 8 - 7 \\ &= -1,\end{aligned}$$

which implies that the ordering $(2,3) \rightarrow (2,2)$ is not feasible. So the disjunctive arc $(2,2) \rightarrow (2,3)$ has to be inserted. In the same way, it can be shown that the disjunctive arcs $(2,2) \rightarrow (2,1)$ and $(1,1) \rightarrow (1,2)$ have to be inserted as well.

Given these additional precedence constraints the release and due dates of all operations have to be updated. The updated release and due dates are presented in the table below.

operations	r_{ij}	d_{ij}
(1,1)	0	18
(2,1)	10	28
(3,1)	18	32
(2,2)	0	18
(1,2)	10	21
(4,2)	13	26
(3,2)	18	32
(1,3)	0	22
(2,3)	8	29
(4,3)	15	32

These updated release and due dates do not imply any additional precedence constraints. Going through Step 5 of the algorithm requires the computation of the factor $\phi'((i,j)(i,k))$ for every unordered pair of operations on each machine.

pair	$\phi'((i,j)(i,k))$
(1,1)(1,3)	$\sqrt{4 \times 8} = 5.65$
(1,2)(1,3)	$\sqrt{5 \times 14} = 8.36$
(2,1)(2,3)	$\sqrt{4 \times 5} = 4.47$
(3,1)(3,2)	$\sqrt{4 \times 4} = 4.00$
(4,2)(4,3)	$\sqrt{3 \times 11} = 5.74$

The pair with the least flexibility is $(3,1)(3,2)$. Since the slacks are such that

$$\sigma_{(3,2)\rightarrow(3,1)} = \sigma_{(3,1)\rightarrow(3,2)} = 4,$$

either precedence constraint can be inserted. Suppose the precedence constraint $(3,2) \rightarrow (3,1)$ is inserted. This precedence constraint causes significant changes in the time windows during which the operations have to be processed.

operations	r_{ij}	d_{ij}
(1,1)	0	14
(2,1)	10	28
(3,1)	24	32
(2,2)	0	14
(1,2)	10	17
(4,2)	13	22
(3,2)	18	28
(1,3)	0	22
(2,3)	8	29
(4,3)	15	32

However, this new set of time windows imposes an additional precedence constraint, namely $(4,2) \rightarrow (4,3)$. This new precedence constraint causes the following changes in the release dates and due dates of the operations.

operations	r_{ij}	d_{ij}
(1,1)	0	14
(2,1)	10	28
(3,1)	24	32
(2,2)	0	14
(1,2)	10	17
(4,2)	13	22
(3,2)	18	28
(1,3)	0	22
(2,3)	8	29
(4,3)	18	32

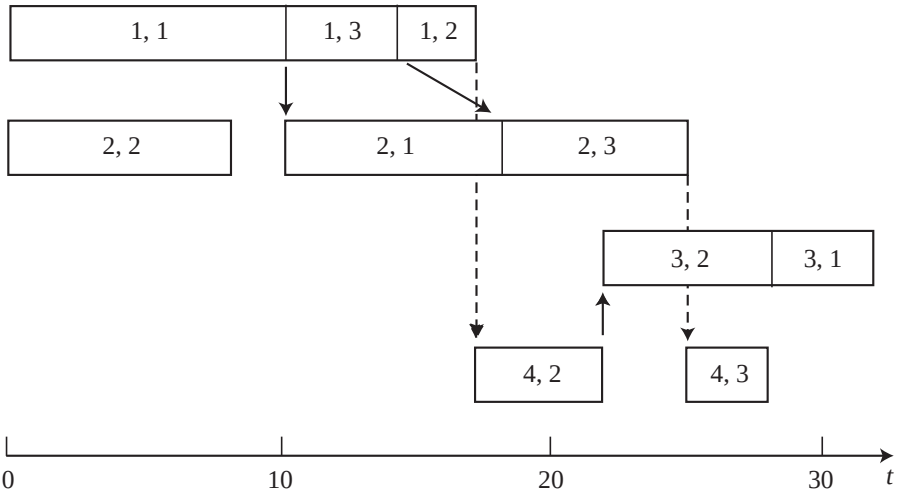

Fig. 5.4. Final schedule in Example 5.5.3.

These updated release and due dates do not imply additional precedence constraints. Going through Step 5 of the algorithm requires the computation of the factor $\phi'((i,j)(i,k))$ for every unordered pair of operations on each machine.

pair	$\phi'((i,j)(i,k))$
(1,1)(1,3)	$\sqrt{0 \times 8} = 0.00$
(1,2)(1,3)	$\sqrt{5 \times 10} = 7.07$
(2,1)(2,3)	$\sqrt{4 \times 5} = 4.47$

The pair with the least flexibility is $(1,1)(1,3)$ and the precedence constraint $(1,1) \rightarrow (1,3)$ has to be inserted.

Inserting this last precedence constraint enforces one more constraint, namely $(2,1) \rightarrow (2,3)$. Now only one unordered pair of operations remains, namely pair $(1,3)(1,2)$. These two operations can be ordered in either way without violating any due dates. A feasible ordering is

$$(1,3) \rightarrow (1,2).$$

The resulting schedule with a makespan of 32 is depicted in Figure 5.4. This schedule meets the due date originally set but is not optimal.

When the pair $(3,1)(3,2)$ had to be ordered, it could have been ordered in either direction because the two slack values were equal. Suppose at that point the opposite ordering was selected, i.e., $(3,1) \rightarrow (3,2)$. Restarting the process at that point yields the following release and due dates.

operations	r_{ij}	d_{ij}
(1,1)	0	14
(2,1)	10	22
(3,1)	18	26
(2,2)	0	18
(1,2)	10	21
(4,2)	13	26
(3,2)	18	32
(1,3)	0	22
(2,3)	8	29
(4,3)	15	32

These release and due dates enforce a precedence constraint on the pair of operations $(2,1)(2,3)$ and the constraint is $(2,1) \rightarrow (2,3)$. This additional constraint has the following effect on the release and due dates:

operations	r_{ij}	d_{ij}
(1,1)	0	14
(2,1)	10	22
(3,1)	18	26
(2,2)	0	18
(1,2)	10	21
(4,2)	13	26
(3,2)	22	32
(1,3)	0	22
(2,3)	18	29
(4,3)	25	32

These new release and due dates have an effect on the pair $(4,2)(4,3)$ and the arc $(4,2) \rightarrow (4,3)$ has to be included. This additional arc does not cause any additional changes in the release and due dates. At this point only two pairs of operations remain unordered, namely the pair $(1,1)(1,3)$ and the pair $(1,2)(1,3)$.

pair	$\phi'((i,j)(i,k))$
(1,1)(1,3)	$\sqrt{0 \times 8} = 0.00$
(1,2)(1,3)	$\sqrt{5 \times 14} = 8.36$

So the pair $(1,1)(1,3)$ is more critical and has to be ordered $(1,1) \rightarrow (1,3)$. It turns out that the last pair to be ordered, $(1,2)(1,3)$, can be ordered either way.

The resulting schedule turns out to be optimal and has a makespan of 28, see Figure 5.5.

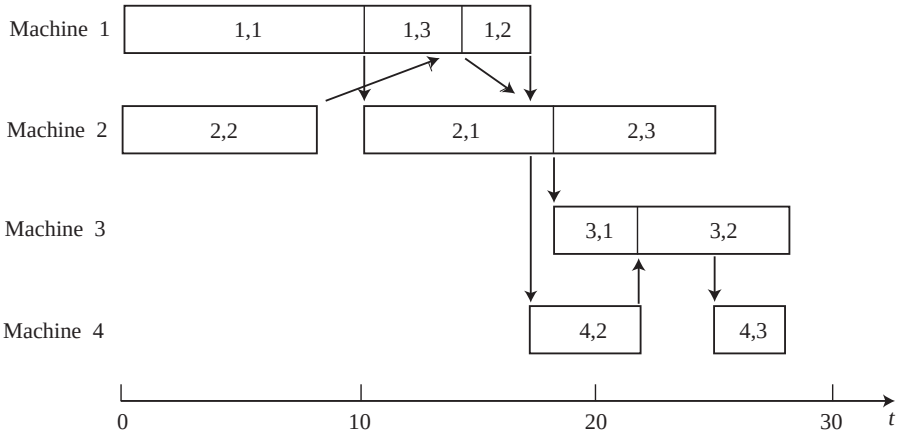

Fig. 5.5. Alternative schedule on Example 5.5.3

As stated before, constraint satisfaction is not only suitable for makespan minimization; it can also be applied to problems with due date related objectives and also when each job has its own release date.

5.6 LEKIN: A Generic Job Shop Scheduling System

Since the job shop is a very common machine environment, many scheduling systems have been developed for this environment. One example of such a system is the LEKIN system. Originally, this system was designed as a tool for teaching and research. Even though the original version was designed with academic goals in mind, several offshoots are being used in real world implementations. The academic version of the system is available on the CD-ROM that is attached to this book.

The system contains a number of scheduling algorithms and heuristics and is designed to allow the user to link and test his or her own heuristics and compare their performances with the heuristics and algorithms that are embedded in the system. The LEKIN system can accomodate various machine environments, namely:

(i) single machine
(ii) parallel machines
(iii) flow shop
(iv) flexible flow shop
(v) job shop
(vi) flexible job shop

Furthermore, it is capable of dealing with sequence dependent setup times in all the environments listed above.

In the main menu the user can select a machine environment. After the user has selected an environment, he has to enter all the necessary machine data and job data manually. However, in the main menu the user also has the option of opening an existing data file. An existing file contains data with regard to one of the machine environments and a specific set of jobs. The user can open an existing file, make changes in the file and work with the modified file. At the end of the session the user can save the modified file under a new name.

If the user wants to enter a data set that is completely new, he first must select a machine environment, and then a dialog box appears where he has to enter the most basic information, i.e., the number of workcenters and the number of jobs to be scheduled. After the user has done this, a second dialog box appears and he has to enter the more detailed workcenter information, i.e., the number of machines at the workcenter, their availability, and the details needed to determine the setup times on each machine (if there are setup times). In the third dialog box the user has to enter the detailed information with regard to the jobs, i.e., release dates, due dates, priorities or weights, routing, and the processing times of the various operations. If the jobs require sequence dependent setup times, then the machine settings that are required for the processing have to be entered. In the LEKIN system the required machine setting is equivalent to a setup time parameter; a sequence dependent setup time is then a function of the parameter of the job just completed and the parameter of the job about to be started.

After all the data have been entered four windows appear simultaneously, namely,

(i) the machine park window,
(ii) the job pool window,
(iii) the sequence window, and
(iv) the Gantt chart window,

(see Figure 5.6).

The machine park window displays all the information regarding the workcenters and the machines. This information is organized in the format of a tree. This window first shows a list of all the workcenters. If the user clicks on a workcenter, the individual machines of that workcenter appear.

The job pool window contains the starting time, completion time, and more information with regard to each job. The information with regard to the jobs is also organized in the form of a tree. First, the jobs are listed. If the user clicks on a specific job, then immediately a list of the various operations that belong to that job appear.

The sequence window contains the lists of jobs in the order in which they are processed on each one of the various machines. The presentation here also has a tree structure. First, all the machines are listed. If the user clicks on a machine, then all the operations that are processed on that machine appear in the sequence in which they are processed. This window is equivalent to the dispatch list interface described in Chapter 13. At the bottom of this

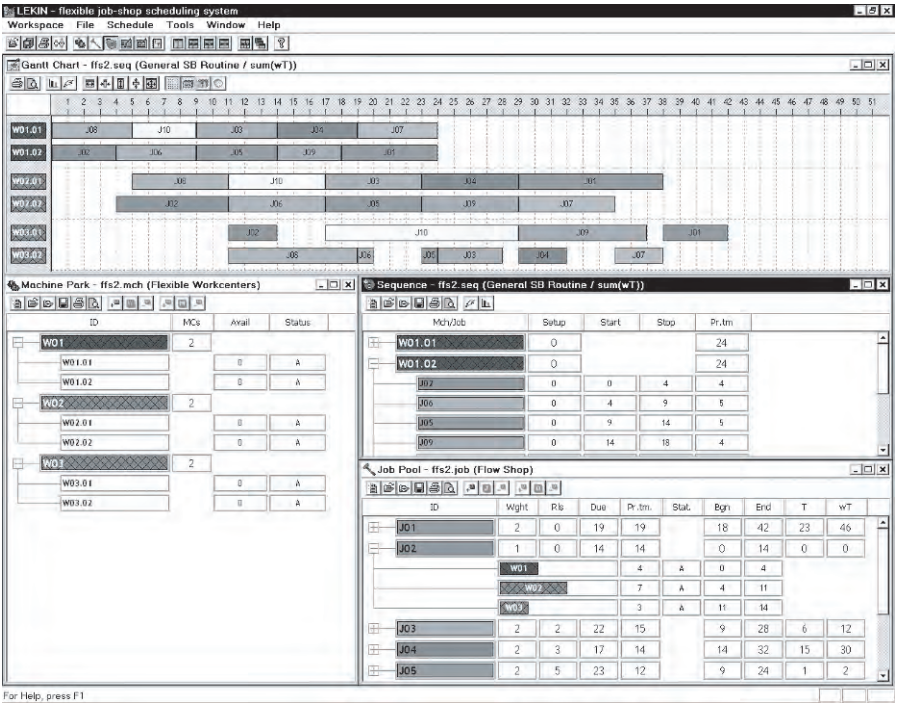

Fig. 5.6. Four windows of the LEKIN system

sequence window there is a summary of the various performance measures of the current schedule.

The Gantt chart window contains a conventional Gantt chart. This Gantt chart window enables the user to do a number of things. For example, the user can click on an operation and a window pops up displaying the detailed information with regard to the corresponding job (see Figure 5.7). The Gantt chart window also has a button that activates a window where the user can see the current values of all the objectives.

The windows described above can be displayed simultaneously on the screen in a number of ways, e.g., in a quadrant style, tiled horizontally, or tiled vertically (see Figure 5.6). Besides these four windows there are two other windows, which will be described in more detail later. These two windows are the logbook window and the objective chart window. The user can print out the windows separately or all together by selecting the print option in the appropriate window.

The data set of a particular scheduling problem can be modified in a number of ways. First, information with regard to the workcenters can be modified in the machine park window. When the user double-clicks on the workcenter the relevant information appears. Machine information can be ac-

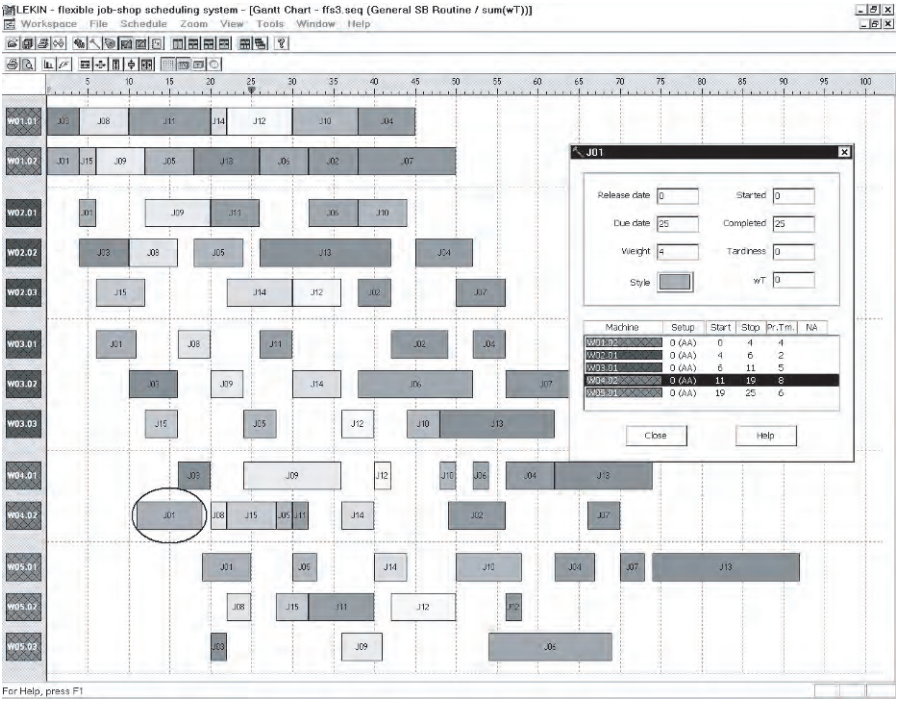

Fig. 5.7. Gantt chart window

cessed in a similar manner. Jobs can be added, modified, or deleted in the job pool window. A double click on a job displays all the relevant information.

After the user has entered the data set, all the information is displayed in the machine park window and job pool window. However, the sequence window and the Gantt chart window remain empty. If the user in the beginning had opened an existing file, then the sequence window and the Gantt chart window may display information pertaining to a sequence that had been generated during an earlier session.

The user can select a schedule from any window. Typically the user would do so either from the sequence window or from the Gantt chart window by clicking on schedule and selecting a heuristic or algorithm from the dropdown menu. A schedule is then generated and displayed in both the sequence window and the Gantt chart window. The schedule generated and displayed in the Gantt chart is a so-called semi-active schedule. A semi-active schedule is characterized by the fact that the start time of any operation of any given job on any given machine is equal to either the completion time of the preceding operation of the same job on a different machine or the completion time of an operation of a different job on the same machine.

The system contains a number of algorithms for several of the machine environments and objective functions. These algorithms include

(i) dispatching rules,
(ii) heuristics of the shifting bottleneck type,
(iii) local search techniques, and
(iv) a heuristic for the flexible flow shop with the total weighted tardiness as objective (SB-LS).

The dispatching rules include EDD and WSPT. The way these dispatching rules are applied in a single machine environment and in a parallel machine environment is standard. However, they can also be applied in the more complicated machine environments such as the flexible flow shop and the flexible job shop. They are then applied as follows: each time a machine is freed the system checks which job should go next on that machine. The system then uses the following data for its priority rules: the due date of a candidate job is then the due date at which the job has to leave the system. The processing time that is plugged in the WSPT rule is the sum of the processing times of all the remaining operations of that job.

The system also has a general purpose routine of the shifting bottleneck type that can be applied to each one of the machine environments and every objective function. Since this routine is quite generic and designed for many different machine environments and objective functions, it cannot compete against a shifting bottleneck heuristic that is tailor-made for a specific machine environment and a specific objective function.

The system also contains a neighbourhood search routine that is applicable to the flow shop and job shop (but not to the flexible flow shop or flexible job shop) with either the makespan or the total weighted tardiness as objective. If the user selects the shifting bottleneck or the local search option, then he must select also the objective he wants to minimize. When the user selects the local search option and the objective, a window appears in which the user has to enter the number of seconds he wants the local search to run.

The system also has a specialized routine for the flexible flow shop with the total weighted tardiness as objective; this routine is a combination of a shifting bottleneck routine and a local search (SB-LS). This routine tends to yield schedules that are of reasonably high quality.

If the user wants to construct a schedule manually, he can do so in one of two ways. First, he can modify an existing schedule in the Gantt chart window as much as he wants by clicking, dragging and dropping operations. After such modifications the resulting schedule is, again, semi-active. However, the user can also construct this way a schedule that is *not* semi-active. To do this he has to activate the Gantt chart, hold down the shift button and move the operation to the desired position. When the user releases the shift button, the operation remains fixed.

A second way of creating a schedule manually is the following. After clicking on schedule, the user must select "manual entry". The user then has to

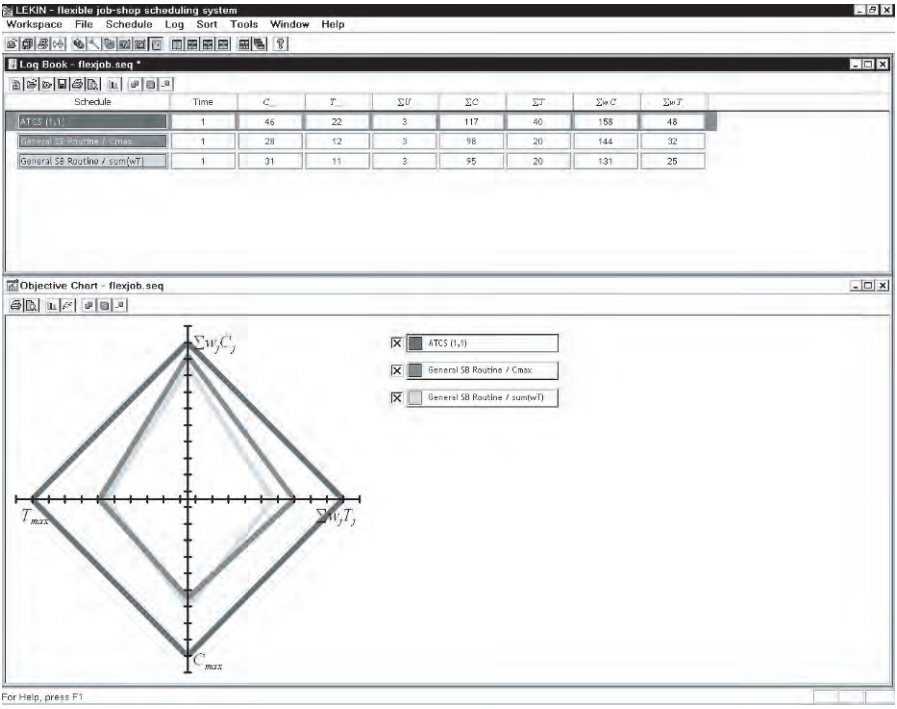

Fig. 5.8. Logbook and comparison of different schedules

enter for each machine a job sequence. The jobs in a sequence are separated from one another by a ";".

Whenever the user generates a schedule for a particular data set, the schedule is stored in a logbook. The system automatically assigns an appropriate name to every schedule generated. The system can store and retrieve a number of different schedules, see Figure 5.8. If the user wants to compare the different schedules he has to click on "logbook". The user can change the name of each schedule and give each schedule a different name for future reference.

The schedules stored in the logbook can be compared with one another by clicking on the "performance measures" button. The user may then select one or more objectives. If the user selects a single objective, a bar chart appears that compares the schedules stored in the logbook with respect to the objective selected. If the user wants to compare the schedules with regard to two objectives, an (x, y) coordinate system appears and each schedule is represented by a dot. If the user selects three or more objectives, then a multi-dimensional graph appears that displays the performance measures of the schedules stored in the logbook.

Some users may want to incorporate the concept of setup times. If there are setup times, then the relevant data have to be entered together with all other

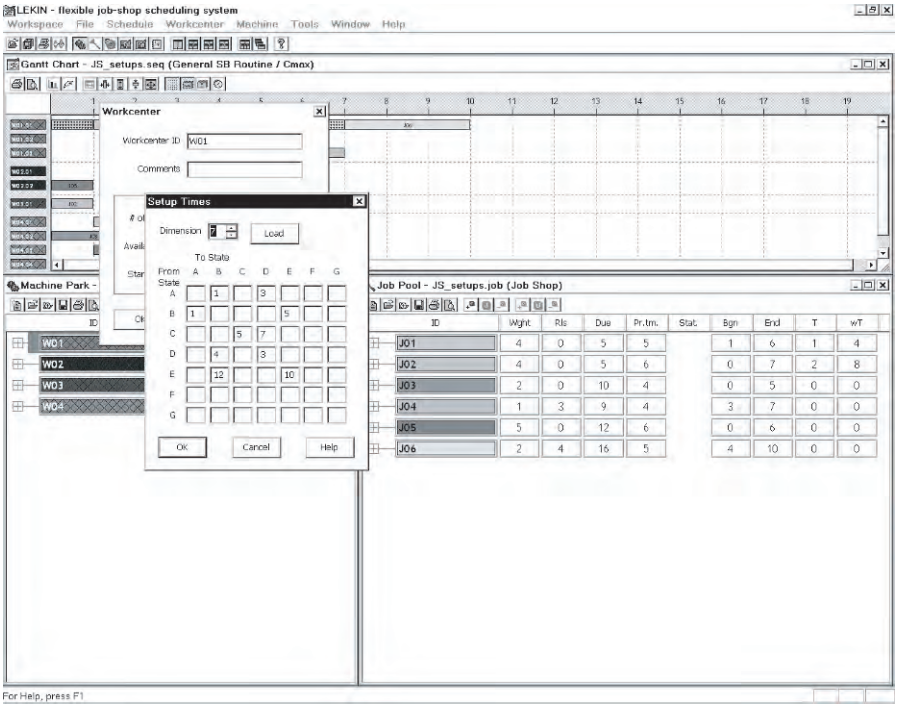

Fig. 5.9. Setup time matrix

job and machine data at the very beginning of a session. (However, if at the beginning of a session an existing file is opened, then such a file may already contain setup data.) The setup times are based on the following structure. Each operation has a single parameter or attribute, which is represented by a letter, e.g., A, B, and so on. This parameter represents the machine setting required for processing that operation. When the user enters the data for each machine, he has to fill in a setup time matrix for that machine. The setup time matrix for a machine specifies the time that it takes to change that machine from one setting to another, i.e., from B to E, see Figure 5.9. The setup time matrices of all the machines at any given workcenter have to be the same (the machines at a workcenter are assumed to be identical). This setup time structure does not allow the user to implement *arbitrary* setup time matrices.

A more advanced user can link his own algorithms to the system. This feature allows the developer of a new algorithm to test his algorithm using the interactive Gantt chart features of the system. The process of making such an external program recognizable by the system consists of two steps, namely, the preparation of the code (programming, compiling and debugging), and the linking of the code to the system.

The linking of an external program is done by clicking on "Tools" and selecting "Options". A window appears with a button for a New Folder and a button for a New Item. Clicking on New folder creates a new submenu. Clicking on a New Item creates a placeholder for a new algorithm. After the user has clicked on a New Item, he has to enter all the data with respect to the New Item. Under "Executable" he has to enter the full path to the executable file of the program. The development of the code can be done in any environment under Win3.2. Appendix F provides examples of the format of a file that contains the workcenter information and the format of a file that contains the information pertaining to the job. After a new algorithm has been added, it is included as one of the heuristics in the schedule option menu.

5.7 Discussion

An enormous amount of research on machine scheduling and job shop scheduling has resulted in many books. This chapter just gives a flavor of the types of results that have appeared in the literature. It focuses mainly on the type of research that has proven to be useful in practice, i.e., decomposition techniques such as the shifting bottleneck heuristic and constraint programming techniques.

A significant amount of more theoretical research has appeared in the literature focusing on exact optimization techniques based on integer programming and disjunctive programming formulations. Some of these formulations are presented in Appendix A. The techniques developed include branch-and-bound as well as branch-and-cut. An example of a branch-and-bound application is presented in Appendix B.

Some of the more applied techniques have not been included in this chapter either. A fair amount of research has been done on local search techniques applied to machine scheduling and job shop scheduling. These techniques include simulated annealing, tabu-search as well as genetic algorithms. Examples of such applications are given in Appendix C.

Appendix D contains a constraint programming formulation in the OPL language of a job shop problem with the makespan objective.

Exercises

5.1. What does the ATC rule reduce to
(a) when K goes to ∞, and
(b) when K is very close to zero?

5.2. Consider an instance with two machines in parallel and the total weighted tardiness as objective to be minimized. There are 5 jobs.

jobs	1	2	3	4	5
p_j	13	9	13	10	8
d_j	6	18	10	11	13
w_j	2	4	2	5	4

(a) Apply the ATC heuristic on this instance with the look-ahead parameter $K = 1$.

(b) Apply the ATC heuristic on this instance with the look-ahead parameter $K = 5$.

5.3. Consider 6 machines in parallel and 13 jobs. The processing times of the 13 jobs are tabulated below.

jobs	1	2	3	4	5	6	7	8	9	10	11	12	13
p_j	6	6	6	7	7	8	8	9	9	10	10	11	11

(a) Compute the makespan under LPT.

(b) Find the optimal schedule.

5.4. Explain why the problem discussed in Section 5.3 is a special case of the problem described in Section 4.6. (*Hint:* Consider an arbitrary job shop scheduling problem and describe it as a workforce constrained project scheduling problem. Consider a machine in the job shop scheduling problem as a workforce pool in the project scheduling problem and the available number in the pool being equal to 1.)

5.5. Consider the following instance of the job shop problem with no recirculation and the makespan as objective.

jobs	*machine sequence*	*processing times*
1	1,2,3	$p_{11} = 9, \quad p_{21} = 8, \quad p_{31} = 4$
2	1,2,4	$p_{12} = 5, \quad p_{22} = 6, \quad p_{42} = 3$
3	3,1,2	$p_{33} = 10, \quad p_{13} = 4, \quad p_{23} = 9$

Give an integer programming formulation of this instance (*Hint:* Consider the integer programming formulation of the project scheduling problem with workforce constraints described in Section 4.6 and Exercise 5.4.)

5.6. Consider the following heuristic for the job shop problem with no recirculation and the makespan objective. Each time a machine is freed, select the job (among those immediately available for processing on the machine) with the longest *total* remaining processing (including its processing on the machine freed). If at any point in time more than one machine is freed, consider first the machine with the largest remaining workload. Apply this heuristic to the instance in Example 5.3.1.

5.7. Apply the heuristic described in Exercise 5.6 to the following instance with the makespan objective.

jobs	*machine sequence*	*processing times*			
1	1,2,3,4	$p_{11} = 9$,	$p_{21} = 8$,	$p_{31} = 4$,	$p_{41} = 4$
2	1,2,4,3	$p_{12} = 5$,	$p_{22} = 6$,	$p_{42} = 3$,	$p_{32} = 6$
3	3,1,2,4	$p_{33} = 10$,	$p_{13} = 4$,	$p_{23} = 9$,	$p_{43} = 2$

5.8. Consider the instance in Exercise 5.7.

(a) Apply the Shifting Bottleneck heuristic to this instance (doing the computation by hand).

(b) Compare your result with the result of the shifting bottleneck routine in the LEKIN system.

5.9. Consider the following two machine job shop with 10 jobs. All jobs have to be processed first on machine 1 and then on machine 2. (This implies that the two machine job shop is actually a two machine flow shop).

jobs	1	2	3	4	5	6	7	8	9	10	11
p_{1j}	3	6	4	3	4	2	7	5	5	6	12
p_{2j}	4	5	5	2	3	3	6	6	4	7	2

(a) Apply the heuristic described in Exercise 5.6 to this two machine job shop.

(b) Construct now a schedule as follows. The jobs have to go through the second machine in the same sequence as they go through the first machine. A job whose processing time on machine 1 is shorter than its processing time on machine 2 must precede each job whose processing time on machine 1 is longer than its processing time on machine 2. The jobs with a shorter processing time on machine 1 are sequenced in increasing order of their processing times on machine 1. The jobs with a shorter processing time on machine 2 follow in decreasing order of their processing times on machine 2. (This rule is usually referred to as *Johnson's Rule.*)

(c) Compare the schedule obtained with Johnson's rule to the schedule obtained under (a).

5.10. Apply the shifting bottleneck heuristic to the two machine flow shop instance in Exercise 5.9. Compare the resulting schedule with the schedules obtained in Exercise 5.9.

Comments and References

Many of the basic priority rules originated in the fifties and early sixties. For example, Smith (1956) introduced the WSPT rule, Jackson (1955) introduced the EDD rule, and Hu (1961) the CP rule. Conway (1965a, 1965b) was one of the first to make

a comparative study of priority rules. A fairly complete list of the most common priority or dispatching rules is given by Panwalkar and Iskander (1977) and a detailed description of composite dispatching rules by Bhaskaran and Pinedo (1992). Special examples of composite dispatching rules are the COVERT rule developed by Carroll (1965) and the ATC rule developed by Vepsalainen and Morton (1987). Ow and Morton (1989) describe a rule for scheduling problems with earliness and tardiness penalties. The ATCS rule is due to Lee, Bhaskaran and Pinedo (1997).

The total weighted tardiness has been the focus of a number of studies in the more basic machine environments, e.g., the single machine. There are many researchers who have dealt with the single machine total weighted tardiness problem; see, for example, Fisher (1976), Potts and Van Wassenhove (1982, 1985, 1987), and Rachamadugu (1987). Abdul-Razaq, Potts, and Van Wassenhove (1990) present a survey of algorithms for the single machine total weighted tardiness problem. Vepsalainen and Morton (1987) developed priority rule based heuristics for job shops with the total weighted tardiness objective.

Job shop scheduling has received an enormous amount of attention in the research literature as well as in books. The special case of the job shop where all the jobs have the same route, i.e., the flow shop, also has received considerable attention. For results on the flow shop, see, for example, Johnson (1954), Palmer (1965), Campbell, Dudek and Smith (1970), Szwarc (1971, 1973, 1978), Gupta (1972), Baker (1975), Smith, Panwalkar and Dudek (1975, 1976), Dannenbring (1977), Muth (1979), Papadimitriou and Kannelakis (1980), Ow (1985), Pinedo (1985), Widmer and Hertz (1989), and Taillard (1990).

An algorithm for the minimization of the makespan in a two machine job shop without recirculation is due to Jackson (1956) and the disjunctive programming formulation described in Section 5.3 is due to Roy and Sussmann (1964).

Branch-and-bound techniques have often been applied to minimize the makespan in job shops; see, for example, Lomnicki (1965), Brown and Lomnicki (1966), McMahon and Florian (1975), Barker and McMahon (1985), Carlier and Pinson (1989), Applegate and Cook (1991), and Hoitomt, Luh and Pattipati (1993). For an overview of branch-and-bound techniques, see Pinson (1995). Singer and Pinedo (1998) developed a branch-and-bound approach for the job shop with the total weighted tardiness objective.

The famous shifting bottleneck heuristic is due to Adams, Balas and Zawack (1988). Their algorithm makes use of a single machine scheduling algorithm developed by Carlier (1982). Earlier work on this particular single machine scheduling problem was done by McMahon and Florian (1975). Nowicki and Zdrzalka (1986), Dauzère-Pérès and Lasserre (1993, 1994) and Balas, Lenstra and Vazacopoulos (1995) all developed more sophisticated versions of the Carlier algorithm. The monograph by Ovacik and Uzsoy (1997) presents an excellent treatise of the application of decomposition methods and shifting bottleneck techniques to large scale job shops with the makespan and the maximum lateness objectives. This monograph is based on a number of papers by the two authors; see, for example, Uzsoy (1993) for the application of decomposition methods to flexible job shops. Pinedo and Singer (1998) developed a shifting bottleneck approach for the job shop problem with the total weighted tardiness objective and Yang, Kreipl, and Pinedo (2000) developed a similar approach for the flexible flow shop with the total weighted tardiness objective.

A fair amount of research has focused on constraint programming approaches for the job shop scheduling problem with the makespan objective; Section 5.5 is

based on the work by Cheng and Smith (1997). Several other researchers have also considered this problem; see, for example, Nuijten and Aarts (1996), and Baptiste, LePape, and Nuijten (2001).

The LEKIN system is due to Asadathorn (1997) and Feldman and Pinedo (1998). The general-purpose routine of the shifting bottleneck type that is embedded in the system is also due to Asadathorn (1997). The local search routines that are applicable to the flow shop and job shop are due to Kreipl (1998). The more specialized SB-LS routine for the flexible flow shop is due to Yang, Kreipl, and Pinedo (2000).

In addition to the procedures discussed in this chapter flow shop and job shop problems have been tackled with local search procedures; see, for example, Matsuo, Suh, and Sullivan (1988), Dell'Amico and Trubian (1991), Della Croce, Tadei and Volta (1992), Storer, Wu and Vaccari (1992), Nowicki and Smutnicki (1996), and Kreipl (1998).

For a broader view of the job shop scheduling problem, see Wein and Chevelier (1992). For an interesting special case of the job shop, i.e., a flow shop with reentry, see Graves, Meal, Stefek and Zeghmi (1983). For results on a generalization of the flow shop, i.e., the flexible flow shop, see Hodgson and McDonald (1981a, 1981b, 1981c).

Of course, many applications of job shop scheduling problems in industry have been discussed in the literature. For job shop scheduling problems and solutions in the micro-electronics industry, see, for example, Wein (1988), Lee, Uzsoy and Martin-Vega (1992) and Uzsoy, Lee and Martin-Vega (1992).

Chapter 6

Scheduling of Flexible Assembly Systems

6.1 Introduction

Flexible assembly systems differ in a number of ways from the job shops considered in the previous chapter. In a job shop, each job has its own identity and may be different from all other jobs. In a flexible assembly system, there are typically a limited number of different product types and the system has to produce a given quantity of each product type. So two units of the same product type are identical.

The movements of jobs in a flexible assembly system are often controlled by a material handling system, which imposes constraints on the starting times of the jobs at the various machines or workstations. The starting time of a job at a machine is a function of its completion time on the previous machine on its route. A material handling system usually also limits the number of jobs waiting in buffers between machines.

In this chapter we analyze three different models for flexible assembly systems. The machine environments in the three models are similar to the machine environments of a flow shop or a flexible flow shop. However, the models tend to be more complicated than the models considered in Chapter 5. This is mainly because of the additional constraints that are imposed by the material handling systems.

M.L. Pinedo, *Planning and Scheduling in Manufacturing and Services*,
DOI: 10.1007/978-1-4419-0910-7_6, © Springer Science + Business Media, LLC 2009

The first model represents a flow line with a number of machines or workstations in series. The line is unpaced, i.e., a machine can spend as much time as needed on any job. There are finite buffers between successive machines which may cause blocking and starving. A number of different product types have to be produced in given quantities and the goal is to maximize the throughput. This type of environment occurs, for example, in the assembly of copiers. Different types of copiers are often assembled on the same line. The different models are usually from the same family and may have many common characteristics. However, they differ with regard to the options. Some types have an automatic document feeder while others have not; some have more elaborate optics than others. The fact that different copiers have different options implies that the processing times at certain stations may vary.

The second model is a paced assembly system with a conveyor system that moves at a constant speed. The units that have to be assembled are moved from one workstation to the next at a fixed speed. Each workstation has its own capacity and constraints. Again, a number of different product types have to be assembled. The goal is to sequence the jobs so that no workstation is overloaded and setup costs are minimized. Paced assembly systems are very common in the automotive industry, where different models have to be assembled on the same line. The cars may have different colors and different option packages. The sequencing of the cars has to take setup costs as well as workload balancing into account.

The third model is a flexible flow system with limited buffers and bypass. In contrast to the first two models, there are a number of machines in parallel at each workcenter. A job may be processed on any one of the parallel machines or it may bypass a workcenter altogether. The objective is to maximize the throughput. This type of system is used, for example, in the manufacturing of Printed Circuit Boards (PCBs). PCBs are produced in batches with each batch requiring its own set of operations.

6.2 Sequencing of Unpaced Assembly Systems

Consider a number of machines in series with a limited buffer between successive machines. The material handling system that moves a job from one machine to the next is unpaced. So whenever a machine finishes processing a job it can release the job to the buffer of the next machine provided that buffer is not full. If that buffer is full, then the machine cannot release the job and is blocked. The material handling system does not permit bypassing, i.e., each machine serves the jobs according to the *First In First Out* discipline. This type of serial processing is common in the assembly of bulky items, such as television sets or copiers; their size makes it difficult to keep many units waiting in front of a machine.

This machine environment with limited buffers is in a mathematical sense equivalent to a system of machines in series with *no* buffers in between ma-

chines. This happens to be the case because a buffer space between two machines may be regarded as a "machine" where the processing times of all jobs happen to be zero. So any number of machines with limited buffers between them can be transformed into an equivalent system that consists of a (larger) number of machines with *zero* buffers in between them. Transforming a model with buffers into an equivalent model without buffers is advantageous, since a model without buffers is easier to describe and to formulate mathematically than a model with buffers. In this section we focus, therefore, on models without buffers; of course, we may have machines where all processing times are zero (playing the role of buffers).

Schedules used in flow lines with blocking are often periodic or cyclic. Such schedules are generated as follows. First, a given set of jobs are scheduled in a certain order. This set contains jobs of all the product types and there may be more than one job of the same product type. This set is followed by a second set that is identical to the first one and scheduled in the same way. This process is repeated over and over again; the resulting schedule is a cyclic schedule.

Cyclic schedules are generally not practical when there are significant sequence dependent setup times between different product types. It is then more efficient to have long runs of the same product type. However, if setup times are negligible, then cyclic schedules have important advantages. For example, if the demand for each product type remains constant over time, then a cyclic schedule results in inventory holding costs that are significantly less than the costs incurred with schedules that have long runs of each product type. Cyclic schedules also have an advantage in that they are easy to keep track of and impose a certain discipline. However, it is not necessarily true that a cyclic schedule has the maximum throughput; often, an acyclic schedule has the maximum throughput. Nevertheless, in practice, a scheduler often adopts a cyclic schedule from which he allows minor deviations, dependent upon current demand.

Suppose there are l different product types. Let $\mathcal{N}_k$ denote the number of jobs of product type k in the overall production target. The production target may be for a period of six months or one year and the numbers may be large. If z is the greatest common divisor of the integers $\mathcal{N}_1, \ldots, \mathcal{N}_l$, then the vector

$$\mathcal{N}^* = \left(\frac{\mathcal{N}_1}{z}, \ldots, \frac{\mathcal{N}_l}{z}\right)$$

represents the smallest set having the same proportions of the different product types as the long range production target. This set is usually referred to as the *Minimum Part Set (MPS)*. Given the vector $\mathcal{N}^*$, the elements in an MPS may be regarded, without loss of generality, as n jobs, where

$$n = \frac{1}{z} \sum_{k=1}^{l} \mathcal{N}_k.$$

Let p_{ij} denote the processing time of job j, $j = 1, \ldots, n$, on machine i. Cyclic schedules are specified by the sequence of the n jobs in the MPS. The fact that various jobs within an MPS may correspond to the same product type and have identical processing requirements does not affect the approach described below. Maximizing system throughput is basically equivalent to minimizing the cycle time of an MPS in a steady state. In this case the MPS cycle time can be defined as the time between the first jobs of two consecutive MPSs entering the system. The following example illustrates the MPS cycle time concept.

Example 6.2.1 (MPS Cycle Time). Consider an assembly system with four machines and no buffers between the machines. There are three different product types that have to be produced in equal amounts, i.e.,

$$\mathcal{N}^* = (1, 1, 1).$$

The processing times of the three jobs in the MPS are:

Jobs	1	2	3
p_{1j}	0	1	0
p_{2j}	0	0	0
p_{3j}	1	0	1
p_{4j}	1	1	0

The second "machine" (that is, the second row of processing times), with zero processing times for all three jobs, functions as a buffer between the first and third machines. The Gantt charts for this example under the two different sequences are shown in Figure 6.1. Under both sequences a steady state is reached during the second cycle. Under sequence $1, 2, 3$ the MPS cycle time is three, while under sequence $1, 3, 2$ the MPS cycle time is two.

For the minimization of the MPS cycle time the so-called *Profile Fitting (PF)* heuristic can be used. This heuristic works as follows: one job is selected to go first. The selection of the first job in the MPS sequence may be done arbitrarily or according to some scheme. For example, one can choose the job with the largest total amount of processing as the one to go first. This first job generates a *profile*. For the time being, we assume that the job does not encounter any blocking and proceeds smoothly from one machine to the next (in a steady state the first job in an MPS may be blocked by the last job from the previous MPS). The profile is determined by the departure time of this first job, say job j_1, from machine i. If X_{i,j_1} denotes the departure (or exit) time of job j_1 from machine i, then

$$X_{i,j_1} = \sum_{h=1}^{i} p_{h,j_1}$$

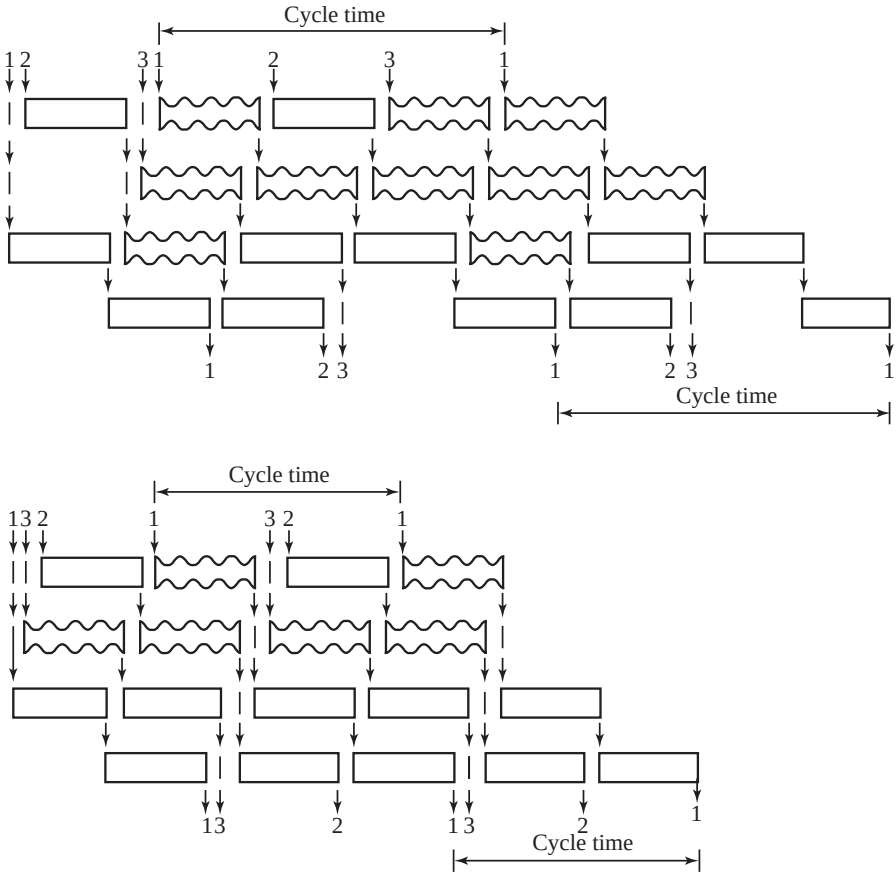

Fig. 6.1. Gantt charts for Example 6.2.1

In order to determine which is the most appropriate job to go second, every one of the remaining jobs in the MPS is tried out. For each candidate job, the amounts of time the machines are idle and the amounts of time the job is blocked at a machine are computed. The departure times of a candidate job for the second position, say job k, can be computed recursively as follows:

$$\begin{aligned} X_{1,j_2} &= \max(X_{1,j_1} + p_{1k}\ ,\ X_{2,j_1}) \\ X_{i,j_2} &= \max(X_{i-1,j_2} + p_{ik}\ ,\ X_{i+1,j_1}), \qquad i = 2, \ldots, m-1 \\ X_{m,j_2} &= X_{m-1,j_2} + p_{mk} \end{aligned}$$

The nonproductive time on machine i, either from being idle or from being blocked, if candidate k is put into the second position, is $X_{i,j_2} - X_{i,j_1} - p_{ik}$. The sum of these idle and blocked times over all m machines is computed for candidate k, i.e.,

$$\sum_{i=1}^{m} \Big(X_{i,j_2} - X_{i,j_1} - p_{ik} \Big).$$

This procedure is repeated for all remaining jobs in the MPS. The candidate with the smallest total amount of nonproductive time is then selected for the second position.

After the best fitting job is added to the partial sequence, the new profile (the departure times of this job from all the machines) is computed and the procedure is repeated. From the remaining jobs in the MPS again the one that fits best is selected. This process continues until all the jobs are scheduled. The PF heuristic functions, in a sense, as a dynamic dispatching rule.

The PF heuristic can be summarized as follows.

Algorithm 6.2.2 (Profile Fitting Heuristic).

Step 1. *(Initial Condition)*

Select the job with the longest total processing time as the first job in the MPS.

Step 2. *(Analysis of Remaining Jobs to be Scheduled)*

For each job that has not yet been scheduled do the following: Consider that job as the next one in the partial sequence and compute the total nonproductive time on all m machines (machine idle time as well as machine blocking time).

Step 3. *(Selection of the Next Job in Partial Schedule)*

Of all jobs analyzed in Step 2, select the job with the smallest total nonproductive time as the next one in the partial sequence.

Step 4. *(Stopping Criterion)*

If all jobs in the MPS have been scheduled, then STOP.

Otherwise, go to Step 2.

Observe that in Example 6.2.1, after job 1, the Profile Fitting heuristic would select job 3 to go next, as this would cause only one unit of blocking time (on machine 2) and no idle times. If job 2 were selected to go after job 1, one unit of idle time would be incurred at machine 2 and one unit of blocking on machine 3, resulting in two units of nonproductive time. So in this example the heuristic would yield an optimal sequence.

Experiments have shown that the PF heuristic results in good schedules. However, it can be refined to perform even better. In the description above, the goodness of fit of a particular job was measured by summing all the nonproductive times on the m machines. Each machine was considered equally important. Suppose one machine is a bottleneck which has more processing to do than any other machine. It is intuitive that lost time on a bottleneck machine is worse than lost time on a machine that, on average, does not have much processing to do. When measuring the total amount of lost

time, it may be appropriate to weight each inactive time period by a factor that is proportional to the degree of congestion at the corresponding machine. The higher the degree of congestion at a machine, the larger the weight. One measure for the degree of congestion of a machine is easy to calculate; simply determine the total amount of processing to be done on all jobs in an MPS at that machine. In the numerical example presented above the third and the fourth machines are more heavily used than the first and second machines (the second machine was not used at all and basically functioned as a buffer). Nonproductive time on the third and fourth machines is therefore less desirable than nonproductive time on the first and second machines. Experiments have shown that such a weighted version of the PF heuristic performs significantly better than the unweighted version.

Example 6.2.3 (Application of Weighted Profile Fitting Heuristic). Consider three machines and an MPS of four jobs. There are no buffers between machines. The processing times of the four jobs on the three machines are as follows.

Jobs	1	2	3	4
p_{1j}	2	4	2	3
p_{2j}	4	4	0	2
p_{3j}	2	0	2	0

All three machines are actual machines and none are buffers. The workloads on the three machines are not entirely balanced. The workload on machine 1 is 11, on machine 2 it is 10 and on machine 3 it is 4. So, time lost on machine 3 is less detrimental than time lost on the other two machines. To apply a weighted version of the Profile Fitting heuristic, the weight given to nonproductive time on machine 3 should be smaller than the weights given to nonproductive times on the other two machines. In this example we set the weights for machines 1 and 2 equal to 1 and the weight for machine 3 equal to 0.

Assume job 1 is the first job in the MPS. The profile can be determined easily. If job 2 is selected to go second, then there will be no idle time or blocking on machines 1 and 2; however machine 3 will be idle for 2 time units. If job 3 is selected to go second, machines 1 and 2 are both blocked for two units of time. If job 4 is selected to go second, machine 1 will be blocked for one unit of time. As the weight of machine 1 is 1 and of machine 3 is 0, the weighted PF selects job 2 to go second. It can be verified that the weighted PF results in the cycle $1, 2, 4, 3$, with an MPS cycle time of 12. If the unweighted version of the PF heuristic were applied and job 1 again was selected to go first, then job 4 would be selected to go second. The unweighted PF heuristic would result in the sequence $1, 4, 3, 2$ with an MPS cycle time of 14.

6.3 Sequencing of Paced Assembly Systems

In a paced assembly line, a conveyor system moves the jobs (e.g., cars) from one workstation to the next at a constant speed. The jobs maintain a fixed distance from one another. If the assembly at a particular station is done manually, the workers walk alongside the moving line while performing their operation; after its completion, they walk back towards that point in the line that corresponds to the beginning of their station or section. The time it takes to walk back is often negligible in comparison with the time it takes to perform the operation. The operations to be done at the various stations are, of course, different and their processing times may not be identical. The space along the line reserved for a particular operation is in proportion to the amount of time needed for that operation. In this type of assembly line a station is basically equivalent to a designated section of the assembly line. At any point in time there may be more than one job at a given station; if that is the case, then several jobs may be processed at that station at the same time. In this environment bypass is not allowed.

An important characteristic of a paced assembly system is its unit cycle time, which is defined as the time between two successive jobs coming off the line. The unit cycle time is the reciprocal of the production rate.

Paced assembly systems are very common in the automotive industry. Assembly lines in the automotive industry have, of course, many other characteristics that are not included in the model described above. One of these is, for example, the point in the line where the engine and the body come together.

In the automotive industry some jobs have due dates. These jobs are made to order and it is important to assemble these in a timely fashion to provide good customer service. However, these committed shipping dates are not as stringent as those in other industries (a job may have to be shipped in a given week, not necessarily on a given day). Other (more standard) jobs are made for inventory and sent to dealers. The target inventory levels of the different models with the various different option packages are determined by the marketing department. This implies that production is a mixture of Make-To-Order and Make-To-Stock. The Make-To-Stock component of the production provides some flexibility for the scheduling system, since the timing of the production of these jobs is somewhat flexible.

Each job that goes through the line has a number of attributes or characteristics such as color, options (e.g., sunroof, power windows), and destination. From a production point of view it is advantageous to sequence jobs that have certain attributes in common one after another. For example, it pays to have a number of consecutive jobs with the same color, since cleaning the paint guns in the paint shop involves a changeover cost. Also, if a number of jobs have to go on the same trailer to the same destination, then these jobs should all appear within the same segment of the sequence. If the first and the last job for one particular trailer are sequenced far apart, then there is a waiting

cost involved. (The destinations that cause problems are usually the points of low demand where only a single trailer goes.) Because of the characteristics that involve changeover costs, a sequencing heuristic has to go through a grouping phase where it attempts to keep jobs with similar attributes together. Grouping may occur with regard to all operations that have significant setup or changeover costs. This does not include only color and destination, it also includes intricate manual assembly, where an immediate repetition has a positive impact on the quality.

There is another class of attributes that have a different (actually, an opposite) effect on the sequencing. Consider the installation of optional equipment such as a sunroof or the special parts that are required in a station wagon. A given percentage of the jobs, say 10%, have to undergo such a special operation. Often, such an operation takes more time than an operation that is common to all jobs; the section of the line assigned to such an operation must therefore be longer than the sections of the line assigned to other operations. As the number of workers assigned to such an operation (e.g., sunroof installation) is proportional to the *average* workload, the jobs that need that operation must be spaced out more or less evenly over the sequence. A heuristic, therefore, has to go through a spacing step that schedules such jobs at regular intervals.

Example 6.3.1 (Sunroof Installation). Suppose the installation of a sunroof lasts four times longer than a typical operation that has to be done on all jobs (e.g., attaching the hood). The section of the line corresponding to the longer operation has to be at least four times longer than the section of the line assigned to the shorter operation. If the section of the line corresponding to the shorter operation contains, on the average, a single job, then the section of the line corresponding to the longer operation contains, on the average, four jobs. If only 10% of the jobs need to undergo the long operation, then, in a perfectly balanced sequence, every tenth job on the line has to undergo the long operation. However, if two consecutive jobs require the installation of a sunroof, then the workers may not be able to complete the work on the second job in time. The reason is the following: they complete the work on the first job when it is about to leave their section and then turn towards the next job and start working on that one. However, this second job is already relatively close to the end of their section and it may not be possible to complete the operation before it leaves their section. If these two particular jobs were spaced more than four positions apart in the sequence, then there would not have been any problem.

This type of operation is usually referred to as a *capacity constrained* operation. For each capacity constrained operation a criticality index can be computed. This criticality index is the minimum number of positions between two jobs requiring the operation divided by the average number of positions between two such jobs. In Example 6.3.1 the index is 4/10. The higher the index, the more critical the operation. Capacity constrained operations require

a careful balancing of the sequence, since proper spacing has a significant impact on quality. This spacing must satisfy the so-called capacity violation constraints; these are basically upper bounds on the frequencies with which jobs may end up in positions where the capacity constrained operations cannot be completed in time.

Based on the considerations described above there are four objectives that have to be taken into account in the sequencing process. One important objective is the minimization of the total setup cost. If in the paint shop a job with color j is followed by a job with color k there is a setup cost c_{jk}. If two consecutive jobs have the same color, then the setup cost is zero. The first objective is to minimize the total setup cost over the entire production horizon.

A second objective concerns the due dates of the Make-To-Order jobs. A job's due date can be translated into a certain position in the sequence; if the job would appear after its "due date position" it would be considered late. The objective to minimize is similar to the total weighted tardiness objective $\sum w_j T_j$. If job j is tardy, its tardiness T_j is measured by the number of positions in between its assigned position and its due date position. In some plants this objective is very important while in other plants it is of no importance (e.g., Cadillacs are more often made to order than Ford Escorts).

A third objective concerns the spacing with regard to the capacity constrained operations. Let $\psi_i(\ell)$ denote the penalty incurred if workstation i has to do work on two jobs that are ℓ positions apart. The penalty $\psi_i(\ell)$ is monotonically decreasing in ℓ and is zero when ℓ is larger than a given $\ell_i^{\min}$. The objective is to minimize the sum of all $\psi_i(\ell)$. This third objective could be considered as part of a more general objective that is described next.

A fourth objective concerns the rate of consumption of material and parts at the workstations. The objective is to keep the consumption rates of all the parts at all the workstations as regular as possible.

The relative weights of the different objectives depend on the specific assembly system and product line. In some assembly lines all jobs are made to stock while in others most jobs are made to order. Some lines have no capacity constrained operations (such as sunroof installation) and the main costs are setup costs (e.g., cleaning paint guns).

One procedure for sequencing paced assembly lines is the so-called *Grouping and Spacing (GS)* heuristic, which has been specifically designed for the paced assembly lines that are common in the automotive industry. In a car assembly facility the number of different models is usually fairly large and there are many different options or option packages that a car can have. Before the heuristic can be applied a detailed analysis has to be done of each operation. Operations with significant setup costs have to be ranked in decreasing order of setup costs and operations with capacity constraints have to be ranked in decreasing order of their criticality indices. The output of this analysis is an input to the heuristic.

The GS heuristic consists of four phases.

(i) Determining the total number of jobs to be scheduled.
(ii) Grouping jobs with regard to operations that have high setup-costs.
(iii) Ordering of the subgroups taking shipping dates into account.
(iv) Spacing jobs within subgroups taking capacity constrained operations into account.

The first phase determines the total number of jobs to be scheduled. This number is usually decided by the scheduler based on his knowledge of the likelihood and the timing of exogenous disruptions that require rescheduling. This number is a trade-off between two factors. A high number allows the scheduler to find a sequence with a lower cost. However, the probability of a disruption is high and if there are disruptions, then the production of some jobs may be postponed unduly. The number to be scheduled typically ranges from the equivalent of one day of production to one week of production.

The second phase forms subgroups taking operations with high setup costs into account. The run lengths of the different colors are determined. These run lengths may be different for different colors. A frequent color, e.g., grey, may have a run length of up to 50, whereas an infrequent color, e.g., purple, may have a run length of 1 or 2. The grouping with regard to destinations is slightly different since the constraints here are less strict. Jobs with the same destination have to be close to one another in the sequence. However, jobs that have to go to other destinations may also be scattered throughout that segment of the sequence. The run lengths determined in this phase are the results of trade-offs between the minimization of setup-costs and the minimization of the holding costs of finished jobs. They also depend somewhat on the committed shipping dates of the Make-To-Order jobs. As such, determining a run length is in a sense similar to computing an Economic Order Quantity (see Chapter 7).

The third phase orders the different subgroups. The order of the different subgroups is mainly determined by the urgency with which the jobs in a group have to be shipped. This urgency is determined by the committed shipping dates of the Make-To-Order jobs as well as the current inventory levels of the Make-To-Stock jobs.

The fourth phase does the internal sequencing within the subgroups considering the capacity constrained operations. It first considers the most critical operation and spaces the jobs that have to undergo this operation as uniformly as possible. Assuming the positions of these jobs as fixed, it then considers the second most capacity constrained operation and spaces out these jobs as uniformly as possible.

Before presenting a numerical example, we describe a simple mathematical model that has many of the characteristics of a paced assembly system. Consider a single machine and n jobs. All the processing times are equal to 1 (since the assembly line moves at a constant speed). Job j has a due date d_j and a weight w_j. Some due dates may be infinite. Job j has l parameters

$a_{j1}, a_{j2}, \ldots, a_{jl}$. The first parameter represents the color, the second one is equal to 1 when the job has a sunroof and 0 otherwise, the third one represents the destination, and so on. If job j is followed by job k and $a_{j1} \neq a_{k1}$ (i.e., the jobs have different colors), then a setup cost c_{jk} is incurred, which is a function of a_{j1} and a_{k1}. If both jobs j and k have sunroofs, i.e., $a_{j2} = a_{k2} = 1$, and they are spaced ℓ positions apart, a penalty $\psi_2(\ell)$ is incurred, which is decreasing in ℓ. If job j is completed after its due date, a weighted tardiness penalty is incurred. The objective is to minimize the total cost, including the setup costs, spacing costs and tardiness costs.

Example 6.3.2 (Application of Grouping and Spacing Heuristic). Consider a single machine and 10 jobs. Each job has unit processing time. Job j has two parameters a_{j1} and a_{j2}. If two consecutive jobs j and k have $a_{j1} \neq a_{k1}$, then a setup cost

$$c_{jk} = \mid a_{j1} - a_{k1} \mid$$

is incurred. If two jobs j and k have $a_{j2} = a_{k2} = 1$ and they are spaced ℓ positions apart (i.e., there are $\ell - 1$ jobs scheduled in between) a penalty cost

$$\psi_2(\ell) = \max(3 - \ell, 0)$$

is incurred. Some jobs have due dates. If job j, with due date d_j, is completed after its due date, a penalty $w_j T_j$ is incurred.

Jobs	1	2	3	4	5	6	7	8	9	10
a_{j1}	1	1	1	3	3	3	5	5	5	5
a_{j2}	0	1	1	0	1	1	1	0	0	0
d_j	∞	2	∞	∞	∞	∞	6	∞	∞	∞
w_j	0	4	0	0	0	0	4	0	0	0

From the data it is clear that there are three groups with regard to the operation with setup costs (e.g., there may be three colors: color 1, color 3, and color 5). The objective is to find the sequence with the minimum total cost.

The first phase of the GS heuristic is not applicable here. There are three groups with regard to attribute 1 (color). Group A consists of jobs 1, 2, and 3, group B consists of jobs 4, 5, and 6, and group C consists of jobs 7, 8, 9, and 10. The best order with regard to setup costs would be A,B,C. However, group C contains a job with due date 6 that would not be completed in time if the groups are ordered that way. Since the tardiness penalty is somewhat high, it may be better to order the groups A,C,B, because in this way job 7 can be completed in time. So the result of the third phase of the GS heuristic is that the groups are ordered in the sequence A,C,B.

The last phase of the GS heuristic considers the capacity constrained operations which are embodied in attribute 2 of this model. The jobs with attribute 2 equal to 1 have to be spaced out as much as possible. It can be verified that

the following sequence minimizes the penalties with regard to the capacity constrained operation: Group A in the order $2, 1, 3$, followed by Group C in the order $8, 7, 9, 10$, and Group B in the order $5, 4, 6$. The sequence of attribute 2 values is then $1, 0, 1, 0, 1, 0, 0, 1, 0, 1$. The total cost with regards to the capacity constrained operation is 3.

Thus, the total cost associated with this sequence is $6 + 3 = 9$ (6 because of setup cost and 3 because of the capacity constrained operation).

The example presented above is clearly very simple. It was not necessary to determine the sizes of each group, since it appeared that there would be just a single group for each value of attribute 1. In larger instances with more jobs, the question of group sizes becomes, of course, a more important issue.

It is not easy to formulate a precise model and algorithm for the paced assembly line sequencing problem in general. The difficulty lies in the various objective functions. The structure of an algorithm depends on the relative importance of each one of the objectives.

Most paced assembly systems in the real world are significantly more complicated than those described in this section. For example, paced assembly lines in the automotive industry may have a resequencing bank immediately after the paint shop that basically partitions the overall problem into two more or less independent subproblems. Another factor is that some cars may need two passes through the paint shop. The grouping and spacing heuristics that have been implemented in industry are, therefore, significantly more complicated than the simple procedure described above.

Paced assembly sequencing problems have also been tackled with constraint programming techniques. Appendix D describes a program for a paced assembly system that is encoded in the constraint programming language OPL.

6.4 Scheduling of Flexible Flow Systems with Bypass

Consider an assembly system with a number of stages in series and, at each stage, a number of machines in parallel. A job, which in this case often is equivalent to a batch of identical items such as Printed Circuit Boards (PCB's), needs processing at every stage, but only on one machine. Usually, any of the machines will do, but it may be the case that at a given stage not all machines are identical and that a given job has to be processed on a certain machine. If a job does not need processing at a stage, the material handling system will allow that job to bypass that stage and all the jobs residing there, see Figure 6.2. A buffer at a stage may have a limited capacity and when a buffer is full, then either the material handling system must come to a standstill, or, if there is an option to recirculate, the jobs must bypass that stage and recirculate. The manufacturing process is repetitive and therefore it is of interest to find a good cyclic schedule (similar to the one in Section 6.2).

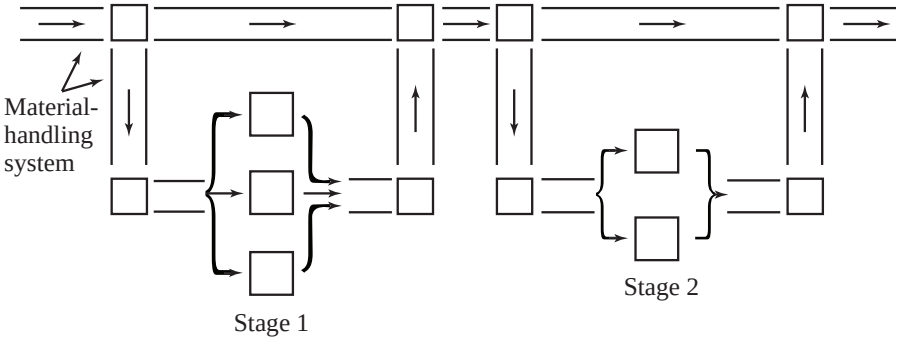

Fig. 6.2. Flexible flow line with bypass

The *Flexible Flow Line Loading (FFLL)* algorithm was designed at IBM for the machine environment described above. It was originally conceived for an assembly system used for the insertion of components in PCB's. The two main objectives of the algorithm are the maximization of throughput and the minimization of WIP. With the goal of maximizing the throughput an attempt is made to minimize the makespan of a whole day's mix. The FFLL algorithm actually attempts to minimize the cycle time of a Minimum Part Set (MPS). (For the definition of the MPS, see Section 6.2.) As buffer spaces are limited, it tries to minimize the WIP to reduce blocking probabilities. The FFLL algorithm consists of three phases:

(i) The machine allocation phase.
(ii) The sequencing phase.
(iii) The release timing phase.

The *machine allocation phase* assigns each job to a particular machine in each bank of machines. Machine allocation is done before sequencing and timing, because, in order to perform the last two phases, the workload assigned to each machine must be known. The lowest conceivable maximum workload for a bank would be obtained if all the machines in a bank were given an equal workload. In order to obtain nearly balanced workloads for the machines at a bank, the Longest Processing Time first (LPT) heuristic is used (see Section 5.2). In this heuristic all the jobs are, for the time being, assumed to be available at the same time and are allocated one at a time to the next available machine in decreasing order of their processing times. After the allocation is determined in this way, the items assigned to a machine may be resequenced; this does not alter the workload balance over the machines at a given bank. The output of this phase is merely the allocation of jobs to machines and not the sequencing of the jobs or the timing of their processing.

The *sequencing phase* determines the order in which the jobs of the MPS are released into the system. This has a significant impact on the MPS cycle

time. The FFLL algorithm uses a so-called *Dynamic Balancing* heuristic for sequencing an MPS. This heuristic is based on the intuition that jobs tend to queue up in the buffer of a machine when a large workload is sent to that machine in a short period of time. This occurs when there is an interval in the loading sequence that contains many jobs with long processing times allocated to one machine. Let n be the number of jobs in an MPS and m the number of machines in the entire system. Let p_{ij} denote the processing time of job j on machine i. Note that $p_{ij} = 0$ for all but one machine in a bank. Let

$$\mathcal{W}_i = \sum_{j=1}^{n} p_{ij}$$

denote the workload in an MPS that is assigned to machine i, and let

$$\mathcal{W} = \sum_{i=1}^{m} \mathcal{W}_i$$

denote the total workload of an MPS. Assuming a fixed sequence, let J_k denote the set of jobs loaded into the system up to and including job k, and let

$$\alpha_{ik} = \sum_{j \in J_k} \frac{p_{ij}}{\mathcal{W}_i}.$$

The α_{ik} represent the fraction of the total workload of machine i that has entered the system by the time job k is loaded. Clearly, $0 \leq \alpha_{ik} \leq 1$. The Dynamic Balancing procedure attempts to keep the $\alpha_{1k}, \alpha_{2k}, \ldots, \alpha_{mk}$ as close to one another as possible, i.e., as close to an ideal target α_k^*, that is defined as

$$\begin{aligned} \alpha_k^* &= \sum_{j \in J_k} \sum_{i=1}^{m} p_{ij} \Big/ \sum_{j=1}^{n} \sum_{i=1}^{m} p_{ij} \\ &= \sum_{j \in J_k} p_j / \mathcal{W}, \end{aligned}$$

where

$$p_j = \sum_{i=1}^{m} p_{ij}$$

is the workload on the entire system due to job j. Hence α_k^* is the fraction of the total system workload that is released into the system by the time job k is loaded. The cumulative workload on machine i, i.e., $\sum_{j \in J_k} p_{ij}$, should be close to the target $\alpha_k^* \mathcal{W}_i$. Now, let o_{ik} denote a measure of *overload* at machine i due to job k entering the system, defined as

$$o_{ik} = p_{ik} - p_k \mathcal{W}_i / \mathcal{W}.$$

Clearly, o_{ik} may be negative, which implies an underload. Let

$$O_{ik} = \sum_{j \in J_k} o_{ij} = \Big(\sum_{j \in J_k} p_{ij} \Big) - \alpha_k^* \mathcal{W}_i$$

denote the cumulative overload (or underload) on machine i due to the jobs in the sequence up to and including job k. To be exactly on target means that machine i is neither overloaded nor underloaded when job k enters the system, i.e., $O_{ik} = 0$. The Dynamic Balancing heuristic attempts to minimize

$$\sum_{k=1}^{n} \sum_{i=1}^{m} \max(O_{ik}, 0).$$

The procedure is basically a greedy heuristic, which selects from among the remaining items in the MPS the one that minimizes the objective at that point in the sequence.

The *release timing phase* works as follows. From the allocation phase the MPS workload at each machine is known. The machine with the greatest MPS workload is the bottleneck since the MPS cycle time cannot be smaller than the MPS workload at the bottleneck machine. We wish to determine a timing mechanism that yields a schedule with a minimum MPS cycle time. First, let all jobs in the MPS enter the system as rapidly as possible. Consider the machines one at a time. At each machine the jobs are processed in the order in which they arrive and processing starts as soon as the job is available. The release times are now modified as follows. Assume that the starting and completion times at the bottleneck machine are fixed. First, consider the machines that are upstream of the bottleneck machine and delay the processing of all jobs on each of these machines, as much as possible, without altering the job sequences. The delays are thus determined by the starting times at the bottleneck machine. Second, consider the machines that are positioned downstream from the bottleneck machine. Process all jobs on these machines as early as possible, again without altering job sequences. These modifications in release times tend to reduce the number of jobs waiting for processing, thus reducing required buffer space.

This three phase procedure attempts to find the cyclic schedule with minimum MPS cycle time in a steady state. If the system starts out empty at some point in time, it may take a few MPS's to reach a steady state. Usually, this transient period is very short. The algorithm tends to achieve short cycle times during the transient period as well.

Extensive experiments with the FFLL algorithm indicates that the method is a valuable tool for the scheduling of flexible flow lines.

Example 6.4.1 (Application of FFLL Algorithm). Consider a flexible flow shop with three stages. At stages 1 and 3 there are two machines in parallel. At stage 2, there is a single machine. There are five jobs in an MPS. Let p'_{hj} denote the processing time of job j at stage h, $h = 1, 2, 3$.

Jobs	1	2	3	4	5
p'_{1j}	6	3	1	3	5
p'_{2j}	3	2	1	3	2
p'_{3j}	4	5	6	3	4

The first phase of the FFLL algorithm performs an allocation procedure for stages 1 and 3. Applying the LPT heuristic to the five jobs on the two machines in stage 1 results in an allocation of jobs 1 and 4 to one machine and jobs 5, 2 and 3 to the other machine. Both machines have to perform a total of 9 time units of processing. Applying LPT to the five jobs on the two machines in stage 3 results in an allocation of jobs 3 and 5 to one machine and jobs 2, 1 and 4 to the other machine (in this case LPT does *not* yield an optimal balance). Note that machines 1 and 2 are at stage 1, machine 3 is at stage 2 and machines 4 and 5 are at stage 3. If p_{ij} denotes the processing time of job j on machine i, we have

Jobs	1	2	3	4	5
p_{1j}	6	0	0	3	0
p_{2j}	0	3	1	0	5
- -	-	-	-	-	-
p_{3j}	3	2	1	3	2
- -	-	-	-	-	-
p_{4j}	4	5	0	3	0
p_{5j}	0	0	6	0	4

The workload $\mathcal{W}_i$ of machine i due to a single MPS can now be computed. The workload vector $\mathcal{W}_i$ is $(9, 9, 11, 12, 10)$ and the entire workload $\mathcal{W}$ is 51. The workload imposed on the entire system due to job j, p_j, can also be computed. The p_j vector is (13, 10, 8, 9, 11). Based on these numbers, all values of o_{ik} can be determined, e.g.,

$$
\begin{aligned}
o_{11} &= 6 - 13 \times 9/51 = +3.71 \\
o_{21} &= 0 - 13 \times 9/51 = -2.29 \\
o_{31} &= 3 - 13 \times 11/51 = +0.20 \\
o_{41} &= 4 - 13 \times 12/51 = +0.94 \\
o_{51} &= 0 - 13 \times 10/51 = -2.55
\end{aligned}
$$

Computing the entire o_{ik} matrix yields:

$$
\begin{Vmatrix}
+3.71 & -1.76 & -1.41 & +1.41 & -1.94 \\
-2.29 & +1.23 & -0.41 & -1.59 & +3.06 \\
+0.20 & -0.16 & -0.73 & +1.06 & -0.37 \\
+0.94 & +2.64 & -1.88 & +0.88 & -2.59 \\
-2.55 & -1.96 & +4.43 & -1.76 & +1.84
\end{Vmatrix}
$$

Of course, the solution also depends on the initial job in the MPS. If the initial job is chosen according to the Dynamic Balancing heuristic, then job 4 goes first and

$$O_{i4} = (+1.41, -1.59, +1.06, +0.88, -1.76).$$

There are four jobs that qualify to go second, namely jobs 1, 2, 3 and 5. If job k goes second, the respective O_{ik} vectors are:

$$O_{i1} = (+5.11, -3.88, +1.26, +1.82, -4.31)$$
$$O_{i2} = (-0.35, -0.36, +0.90, +3.52, -3.72)$$
$$O_{i3} = (+0.00, -2.00, +0.33, -1.00, +2.67)$$
$$O_{i5} = (-0.53, +1.47, +0.69, -1.71, +0.08)$$

The dynamic balancing heuristic then selects job 5 to go second. Proceeding in the same manner the dynamic balancing heuristic selects job 1 to go third and

$$O_{i1} = (+3.18, -0.82, +0.89, -0.77, -2.47).$$

Proceeding, we see that job 3 goes fourth. Then

$$O_{i3} = (+1.76, -1.23, +0.16, -2.64, +1.96).$$

The final cycle is thus $4, 5, 1, 3, 2$.

Applying the release timing phase to this cycle results initially in the schedule presented in Figure 6.3. The MPS cycle time of 12 is actually determined by machine 4 (the bottleneck machine) and there is therefore no idle time allowed between the processing of jobs on this machine. It is clear that the processing of jobs 3 and 2 on machine 5 can be postponed by three time units.

The model discussed in this section can be compared to a flexible flow shop (which is a special case of a flexible job shop). There are similarities as well as differences between the model considered in this section and a flexible flow shop that fits within the framework of Chapter 5. The machine environments in the two models are similar, since both settings are flexible flow shops that allow bypass. In the model described in this section there is the material handling system that imposes additional constraints that have an impact on the objectives. The limited buffers in the flexible assembly system require a minimization of the Work-In-Process. The objectives in the two models are also different from another point of view. The models described in Chapter 5 tend to be more Make-To-Order; an important objective in such models may be the minimization of the total weighted tardiness. The model considered in this section is basically a Make-To-Stock model. The main objective is the maximization of throughput.

6.5 Mixed Model Assembly Sequencing at Toyota

Among the major car manufacturers Toyota has been always one of the more innovative ones as far as manufacturing and assembly is concerned. Toyota

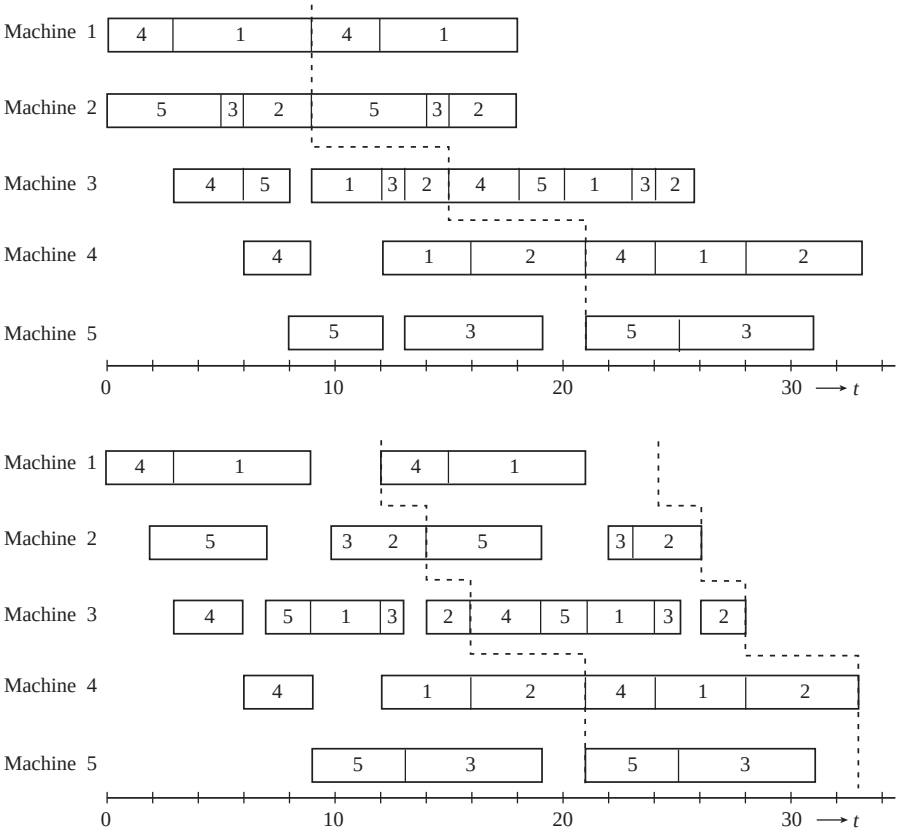

Fig. 6.3. Gantt charts for the FFLL algorithm

operates according to the Just-In-Time (JIT) principle. To make the JIT operation run smoothly, it is important that the consumption rates of all the parts at each station are kept as regular as possible. Toyota's most important goal in the operation of its mixed model assembly lines is to keep the rates of consumption of all parts more or less constant. In other words, the quantity of each part used per hour must be maintained as regular as possible.

As described in the section on paced assembly systems, balancing the workload at each one of the workstations over time is often an important objective in car assembly. Some cars may need more than the average amount of processing at a particular workstation, while others may need less. However, if Toyota manages to keep the consumption of all the parts at each one of the workstations more or less constant, then the workloads are balanced as well.

In order to formalize the Toyota objective, let $\mathcal{N}$ denote the total number of cars to be sequenced. This number is a measure of the planning horizon. Let ℓ denote the number of different models and $\mathcal{N}_j$, $j = 1, \ldots, \ell$, the number of

units of model j that have to be produced over the period under consideration. Thus

$$\mathcal{N} = \sum_{j=1}^{\ell} \mathcal{N}_j.$$

In practice the planned production quantity may be around 500 and the number of different models around 180. Let ν denote the number of different types of parts needed by all workstations for the assembly of the $\mathcal{N}$ cars and let b_{ij}, $i = 1, \ldots, \nu$, $j = 1, \ldots, \ell$, denote the number of parts of type i needed for the assembly of one unit of model j. Let R_i denote the total number of parts of type i required for the assembly of all $\mathcal{N}$ cars and x_{ik} the total number of parts of type i needed in the assembly of the first k units in the sequence. So $R_i/\mathcal{N}$ is the average number of part i required for the assembly of a car and $kR_i/\mathcal{N}$ the average number of part i required for the assembly of k cars.

To keep the rate of consumption of part i as regular as possible, the variable x_{ik} should be as close as possible to the value $kR_i/\mathcal{N}$. Consider now all ν parts and the two vectors

$$\Big(\frac{kR_1}{\mathcal{N}}, \ldots, \frac{kR_\nu}{\mathcal{N}}\Big)$$

and

$$(x_{1k}, \ldots, x_{\nu k}).$$

The first vector plays the role of the target or goal, while the second vector represents the actual numbers of parts needed for the assembly of the first k units in the given sequence. Let

$$\Delta_k = \sqrt{\sum_{i=1}^{\nu} \Big(\frac{kR_i}{\mathcal{N}} - x_{ik}\Big)^2}$$

denote a measure of the difference between the two vectors. Let $\Delta_k(j)$ denote the value of this difference if model j has been put in the k-th position, i.e.,

$$\Delta_k(j) = \sqrt{\sum_{i=1}^{\nu} \Big(\frac{kR_i}{\mathcal{N}} - x_{i,k-1} - b_{ij}\Big)^2}.$$

In order to minimize this objective, Toyota developed the Goal Chasing method which can be described as follows.

Algorithm 6.5.1 (Goal Chasing Method).

Step 1.

Set $x_{i0} = 0$, $S_0 = \{1, 2, \ldots, \ell\}$, *and* $k = 1$.

Step 2.

Select for the k*-the position in the sequence model* j^* *that minimizes the measure* $\Delta_k(j)$, *i.e.,*

$$\Delta_k(j^*) = \min_{j \in S_{k-1}} \left\{ \sqrt{\sum_{i=1}^{\nu} \left(\frac{kR_i}{\mathcal{N}} - x_{i,k-1} - b_{ij} \right)^2} \right\},$$

Step 3.

If more units of model j^* remain to be sequenced, set $S_k = S_{k-1}$

If all units of model j^* have now been sequenced, set $S_k = S_{k-1} - \{j^*\}$.

Step 4.

If $S_k = \emptyset$, then STOP.

If $S_k \neq \emptyset$, set $x_{ik} = x_{i,k-1} + b_{ij^*}, \quad i = 1, \ldots, \nu$.

Set $k = k + 1$ and go to Step 2.

Since the number of parts in a car is around 20,000, it is in practice difficult to apply the Goal Chasing method to all parts. Therefore, the parts are represented only by their respective subassembly. The number of subassemblies is around 20 and Toyota gives the important subassemblies additional weight. The subassemblies include:

(i) engines,	(vi) bumpers,
(ii) transmissions,	(vii) steering assemblies,
(iii) frames,	(viii) wheels,
(iv) front axles,	(ix) doors, and
(v) rear axles,	(x) air conditioners.

The fact that the consumption rates of the parts are the main objective gives an indication of how important JIT is for Toyota.

Note the similarity between the Goal Chasing method and the Dynamic Balancing routine described in Section 6.4. The physical environment at Toyota is, of course, more similar to the environment of the paced assembly systems described in Section 6.3.

6.6 Discussion

Scheduling problems in flexible assembly systems usually are not as easy to formulate as the more conventional job shop scheduling problems. The material handling systems or conveyor systems impose constraints that tend to be application-specific and difficult to formulate mathematically. Since these problems are hard to formulate as mathematical programs the solution procedures are usually not based on branch-and-bound, but rather based on heuristics. The framework of a heuristic is typically modular and application-specific. Other approaches include constraint programming techniques. Appendix D presents a constraint programming application of a mixed model assembly line.

Another class of systems that share many of the scheduling complexities of flexible assembly systems are the flexible manufacturing systems. In general, a flexible manufacturing system may be defined as a production system that is capable of producing a variety of part types; it consists of (numerically controlled) machines that are connected to one another by an automated material handling system. The entire system typically is under computer control. With the appropriate tool setups, each machine in the system can perform different operations. An operation may therefore be performed at any one of a number of machines. The routing of a job through the system is therefore flexible and is a type of decision that has to be made in the scheduling process. Two other types of decisions are the sequencing of the operations on the machines and the setups of the tools on the machines. These scheduling problems are harder than the classical job shop scheduling problems and the scheduling problems that occur in flexible assembly systems. As in flexible assembly systems, material handling systems and limited buffers increase the complexity. Just as a flexible assembly system often can be viewed as a (flexible) flow shop with a number of additional constraints, a flexible manufacturing system can be viewed as a (flexible) job shop with a number of additional constraints. While a limited buffer in a flexible assembly system may cause blocking, a limited buffer in a flexible manufacturing system may not only cause blocking but also deadlock.

The scheduling problems in flexible manufacturing systems are not easy to formulate as mathematical programs when all decisions and constraints have to be included in the formulation. Even when they can be formulated as mathematical programs, they are still very hard to solve. Many formulations therefore incorporate only one or two of the three types of decisions listed above and ignore some of the resource constraints.

Most of the research in flexible assembly systems and flexible manufacturing systems have focused on specific problems arising in industry. While a classification scheme has been available for job shop scheduling problems for quite some time, it is only recently that such a scheme has been emerging for scheduling problems in flexible manufacturing systems.

Exercises

6.1. Consider in the model of Section 6.2 an MPS of 5 jobs.

(a) Show that when all jobs are different the number of different cyclic schedules is 4!.

(b) Compute the number of different cyclic schedules when two jobs are the same (i.e., there are four different job types among the 5 jobs).

6.2. Consider the model discussed in Section 6.2 with 4 machines and an MPS of 4 jobs.

Jobs	1	2	3	4
p_{1j}	6	4	6	8
p_{2j}	2	10	4	6
p_{3j}	4	8	0	2
p_{4j}	8	2	6	6

(a) Apply the unweighted PF heuristic to find a cyclic schedule. Choose job 1 as the initial job and compute the MPS cycle time.

(b) Apply again the unweighted PF heuristic. Choose job 2 as the initial job and compute the MPS cycle time.

(c) Find the optimal schedule.

6.3. Consider the same problem as in the previous exercise.

(a) Apply a weighted PF heuristic to find a cyclic schedule. Choose the weights associated with machines $1, 2, 3, 4$ as $2, 2, 1, 2$, respectively. Select job 1 as the initial job.

(b) Apply again a weighted PF heuristic but now with weights $3, 3, 1, 3$. Select again job 1 as the initial job.

(c) Repeat again (a) and (b) but select job 2 as the initial job.

(d) Compare the impact of the weights on the heuristic's performance with the impact of the selection of the first job on the performance.

6.4. Consider the model discussed in Section 6.2. Assume that a system is in a steady state if each machine is in a steady state. That is, at each machine the departure of job j in one MPS occurs exactly the cycle time before the departure of job j in the next MPS. Construct an example with 3 machines and an MPS of a single job that takes more than 100 MPS's to reach a steady state, assuming the system starts out empty.

6.5. In order to model a paced assembly line consider a single machine and n jobs. The processing times of each one of the jobs is 1. Job j has a due date d_j and a weight w_j. If job j is completed after its due date, then a penalty $w_j T_j$ is incurred. There are sequence dependent setup costs c_{jk} but no setup times. The sequence dependent setup costs are determined as follows: job j has an attribute a_j that can be either 0 or 1. Most jobs have an attribute value a_j equal to 0. If there are two jobs with a_j equal to 1 sequenced in such a way that there are less than 3 jobs with a_j equal to 0 in between, then a penalty cost $c_1 = 3$ is incurred. (This implies that this attribute a_j corresponds to a capacity constrained operation).

(a) Design an algorithm for finding a sequence with minimum cost. (*Hint:* Consider, for example, a composite dispatching rule followed by a local search; see Appendix C.)

(b) Apply the algorithm developed to the following instance.

Jobs	1	2	3	4	5	6	7	8	9	10
a_j	0	1	1	0	0	0	1	0	0	0
d_j	∞	2	∞	1	∞	5	6	∞	2	∞
w_j	0	4	0	1	0	3	4	0	3	0

Compute the total cost of the sequence.

6.6. Apply the GS heuristic to the instance in Exercise 6.5. Compare the result with that obtained in the previous exercise.

6.7. Consider the model in Exercise 6.5. Now job j has two attributes a_j and b_j. The a_j attribute is the same as in Exercise 6.5. The b_j values also can be either 0 or 1. If two jobs with b_j value equal to 1 are positioned in such a way that there are 4 or less jobs with b_j value equal to 0 in between, then a penalty cost $c_2 = 3$ is incurred. (This situation corresponds to a line with two capacity constrained operations).

(a) Describe how the algorithm of Exercise 6.5 has to be modified to take this generalization into account.

(b) Apply your algorithm to the instance below.

Jobs	1	2	3	4	5	6	7	8	9	10
a_j	0	1	1	0	0	0	1	0	0	0
b_j	0	1	0	1	1	0	1	0	0	0
d_j	∞	2	∞	1	∞	5	6	∞	2	∞
w_j	0	4	0	1	0	3	4	0	3	0

Compute the total cost of the sequence.

6.8. Consider the application of the FFLL algorithm in Example 6.4.1. Instead of letting the dynamic balancing heuristic minimize

$$\sum_{i=1}^{m}\sum_{k=1}^{n}\max(O_{ik},0),$$

let it minimize

$$\sum_{i=1}^{m}\sum_{k=1}^{n}\mid O_{ik}\mid .$$

Redo Example 6.4.1 and compare the performances of the two dynamic balancing heuristics.

6.9. Consider the application of the FFLL algorithm to the instance in Example 6.4.1. Instead of applying LPT in the first phase of the algorithm, find the optimal allocation of jobs to machines (which leads to a perfect balance of machines 4 and 5). Proceed with the sequencing phase and release timing phase based on this new allocation.

6.10. Consider the instance in Example 6.4.1 again.

(a) Compute in Example 6.4.1 the number of jobs waiting for processing at each stage as a function of time and determine the required buffer size at each stage.

(b) Consider the application of the FFLL algorithm to the instance in Example 6.4.1 with the machine allocation as prescribed in Exercise 6.9. Compute the number of jobs waiting for processing at each stage as a function of time and determine the required buffer size.

(Note that with regard to the machines before the bottleneck, the release timing phase in a sense postpones the release of each job as much as possible and tends to reduce the number of jobs waiting for processing at each stage.)

Comments and References

Flexible assembly systems constitute a subcategory of flexible manufacturing systems (FMS). (Another subcategory of FMS are the general flexible machining systems (GFMS).) A significant amount of research has been done on scheduling in FMS in general. For an overview of scheduling research in FMS, see MacCarthy and Liu (1993), Rachamadugu and Stecke (1994), and Basnet and Mize (1994). Liu and MacCarthy (1996) clarify the basic concepts of scheduling in FMS and classify these scheduling problems according to a classification scheme that is based on the configurations of these systems. For a comprehensive mixed integer linear programming model for the basic scheduling problem that occurs in an FMS, see Liu and MacCarthy (1997). The scheduling of a single flexible machine is studied by Tang and Denardo (1988).

A certain amount of research has been done on the scheduling of flexible assembly systems. The unpaced assembly system with limited buffers is analyzed by Pinedo, Wolf and McCormick (1986), McCormick, Pinedo, Shenker and Wolf (1989), and Abadie, Hall and Sriskandarajah (2000). Its transient analysis is discussed in McCormick, Pinedo, Shenker and Wolf (1990). For more results on cyclic scheduling, see Matsuo (1990) and Roundy (1992), and for more results on flow shops with limited buffers, see Wismer (1972) and Pinedo (1982). Hall and Sriskandarajah (1996) present a survey that includes many of these cyclic scheduling models with limited buffers. For the scheduling of a robotic assembly system, see Hall, Kamoun and Sriskandarajah (1997).

The book by Scholl (1998) focuses on the balancing and sequencing of assembly lines (paced assembly systems). For scheduling problems and solutions in the automotive industry, see, for example, Burns and Daganzo (1987), Yano and Bolat (1989), Bean, Birge, Mittenthal and Noon (1991), Garcia-Sabater (2001) and Drexl and Kimms (2001). Important aspects of scheduling problems in the automotive industry are the sequence dependent setup costs and setup times. The CD-ROM that is attached to this book contains several mini-cases that focus on assembly line sequencing in the automotive industry (projects at Nissan and Peugeot).

A considerable amount of research has focused specifically on sequence dependent setups; see, for example, Gilmore and Gomory (1964), and Bianco, Ricciardelli, Rinaldi and Sassano (1988). Crama, Kolen and Oerlemans (1990) develop a compre-

hensive hierarchical decision model for planning a multiple machine flow line with sequence dependent setups.

The scheduling problem concerning the flexible flow line with limited buffers and bypass is based on a setting found at IBM and analyzed by Wittrock (1985, 1988, 1990).

The way Toyota schedules its assembly lines is discussed in Monden (1983); Section 6.5 is based on Monden's Appendix 2.

Chapter 7

Economic Lot Scheduling

7.1 Introduction

In a job shop, each job has its own identity and its own set of processing requirements. In a flexible assembly system, there are a number of different types of jobs and jobs of the same type have identical processing requirements; in such a system, setup times and setup costs are often not important and a schedule may alternate many times between jobs of different types. In a flexible assembly system an alternating schedule is often more efficient than a schedule with long runs of identical jobs.

In the models considered in this chapter, a set of identical jobs may be large and setup times and setup costs between jobs of two different types may be significant. A setup typically depends on the characteristics of the job about to be started and the one just completed. If a job's processing on a machine requires a major setup then it may be advantageous to let this job be followed by a number of jobs of the same type.

In this chapter we refer to jobs as items and we call the uninterrupted processing of a series of identical items a run. If a facility or machine is geared to produce identical items in long runs, then the production tends to be Make-To-Stock, which inevitably involves inventory holding costs. This

M.L. Pinedo, *Planning and Scheduling in Manufacturing and Services,*
DOI: 10.1007/978-1-4419-0910-7_7, © Springer Science + Business Media, LLC 2009

form of production is, at times, also referred to as continuous manufacturing (in contrast to the forms of discrete manufacturing considered in the previous chapters). The time horizon in continuous manufacturing is often in the order of months or even years. The objective is to minimize the total cost, which includes inventory holding cost as well as setup cost. The optimal schedule is typically a trade-off between inventory holding costs and setup costs and is often repetitive or cyclic.

The associated scheduling problem has several aspects. First, the lengths of the runs have to be determined and, second, the order of the different runs has to be established. The run lengths are typically referred to as the lot sizes and they are the result of trade-offs between setup costs and inventory holding costs. The lots have to be sequenced in such a way that the setup times and setup costs are minimized. This scheduling problem is referred to as the *Economic Lot Scheduling Problem (ELSP)*.

In the standard ELSP a single facility or machine has to produce n different items. The machine can produce items of type j at a rate of Q_j per unit time. If an item of type j is regarded as a job with processing time p_j, then $Q_j = 1/p_j$. We assume that the demand rate for type j is constant at D_j items per unit time. The inventory holding cost for one item of type j is h_j dollars per unit time. If an item of type j is followed by an item of type k a setup cost c_{jk} is incurred; moreover, a setup time s_{jk} may be required. In some models we assume that a setup involves a cost but no machine time and in other, more general, models we assume that a setup involves a cost as well as machine time. The setup cost and time may be either sequence dependent or independent. If the setup cost (time) is sequence independent, then $c_{jk} = c_k$ ($s_{jk} = s_k$). The problem can be viewed as one of deciding a cycle length x and a sequence of runs or cycle $j_1, j_2, \ldots, j_\nu$. This sequence may contain repetitions, so $\nu \geq n$. The associated run times are $\tau_{j_1}, \tau_{j_2}, \ldots, \tau_{j_\nu}$ and there may be idle time between two consecutive runs.

In practice, there are many applications of economic lot scheduling. In the process industries (e.g., the chemical, paper, pharmaceutical, aluminum and steel industries) setup costs and inventory holding costs are significant. When minimizing the total costs, a scheduling problem often reduces to an economic lot scheduling problem (see Example 1.1.4). There are applications of lot scheduling in the service industries as well. In the retail industry (e.g., Sears, Wal-Mart) the procurement of each item has to be controlled carefully. Placing an order for additional supplies entails an ordering cost and keeping a supply in inventory entails a holding cost. The retailer has to determine the trade-off between the inventory holding costs and the ordering costs.

7.2 One Type of Item and the Economic Lot Size

In this section we consider the simplest case, namely a single machine and one type of item. Since there is only one type of item the subscript j can

be dropped, i.e., the production rate is Q and the demand rate is D items per unit time. We assume that the machine capacity is sufficient to meet the demand, i.e., $Q > D$. The problem is to determine the length of a production run. After a run has been terminated and sufficient inventory has been built up, the machine remains idle until the inventory has been depleted and a new run is about to start. Clearly, the length of a production run is determined by the trade-off between inventory holding costs and setup costs. In order to minimize the total cost per unit time we have to find an expression for the total cost over a cycle.

Let x denote the cycle time that has to be determined. If D denotes the demand rate, then the demand over a cycle is Dx and the length of a production run to meet the demand over a cycle is Dx/Q. If the inventory level at the beginning of the production run is zero, then the inventory level goes up during the run at a rate $Q - D$ until it reaches

$$(Q - D)\frac{Dx}{Q}.$$

During the idle period the inventory level goes down at a rate D until it reaches zero and the next production run starts. So the average inventory level is

$$\frac{1}{2}\Big(Dx - \frac{D^2x}{Q}\Big).$$

Each production run incurs a setup cost c. The average cost per unit time due to setups is therefore c/x. Let h denote the inventory holding cost per item per unit time. The total average cost per unit time due to inventory holding costs and setups is therefore

$$\frac{1}{2}h\Big(Dx - \frac{D^2x}{Q}\Big) + \frac{c}{x}.$$

To determine the optimal cycle length we take the derivative of this expression with respect to x and set it equal to zero, yielding

$$\frac{1}{2}hD\Big(1 - \frac{D}{Q}\Big) - \frac{c}{x^2} = 0.$$

Straightforward algebra gives the optimal cycle length

$$x = \sqrt{\frac{2\,Q\,c}{hD(Q - D)}}.$$

The total amount to be produced during a cycle, i.e., the lot size, is

$$Dx = \sqrt{\frac{2\,DQ\,c}{h(Q - D)}}.$$

The lot size Dx is not necessarily an integer number. (This is one of the differences between continuous models and discrete models; this difference is examined more closely in Example 7.2.2.)

The idle time of the machine during a cycle turns out to be

$$x\Big(1 - \frac{D}{Q}\Big).$$

The ratio D/Q is at times denoted by ρ, and may be regarded as the utilization of the machine, i.e., the proportion of time that the machine is busy.

Now consider the limiting case when the production rate Q is arbitrarily high, i.e., $Q \to \infty$. Then,

$$x = \lim_{Q\to\infty} \sqrt{\frac{2\,Q\,c}{hD(Q-D)}} = \sqrt{\frac{2c}{hD}}.$$

In this case the lot size is equal to

$$Dx = \sqrt{\frac{2Dc}{h}},$$

which is often called the *Economic Lot Size (ELS)* or *Economic Order Quantity (EOQ).*

All the expressions above are based on the assumption that there is a setup cost but not a setup time. If, in addition to the setup cost, there is also a setup time s and $s \le x(1-\rho)$, then the solution presented above is still feasible and optimal. If

$$s > x(1-\rho),$$

then the lot size computed above is infeasible. The optimal solution then is the solution where the machine alternates between setups and production runs with a cycle length

$$x = \frac{s}{1-\rho}.$$

That is, the machine is either producing or being set up for the next run. The machine is never idle.

The first example illustrates the use of these formulae.

Example 7.2.1 (The ELSP with and without Setup Times). Consider a facility with a production rate $Q = 90$ items per week, a demand rate $D = 50$ items per week, a setup cost $c = \$2000$, a holding cost $h = \$20$ per item per week, and no setup times. From the analysis above it follows that the cycle time x is 3 weeks and the quantity produced in a cycle is 150. Figure 7.1.a depicts the inventory level over the cycle. The idle time during a cycle is $3(1 - 5/9) = 1.33$ weeks, which is approximately 9 days.

Now suppose that there are setup times. If the setup time is less than 9 days (the length of the idle period), then the 3 week cycle remains optimal.

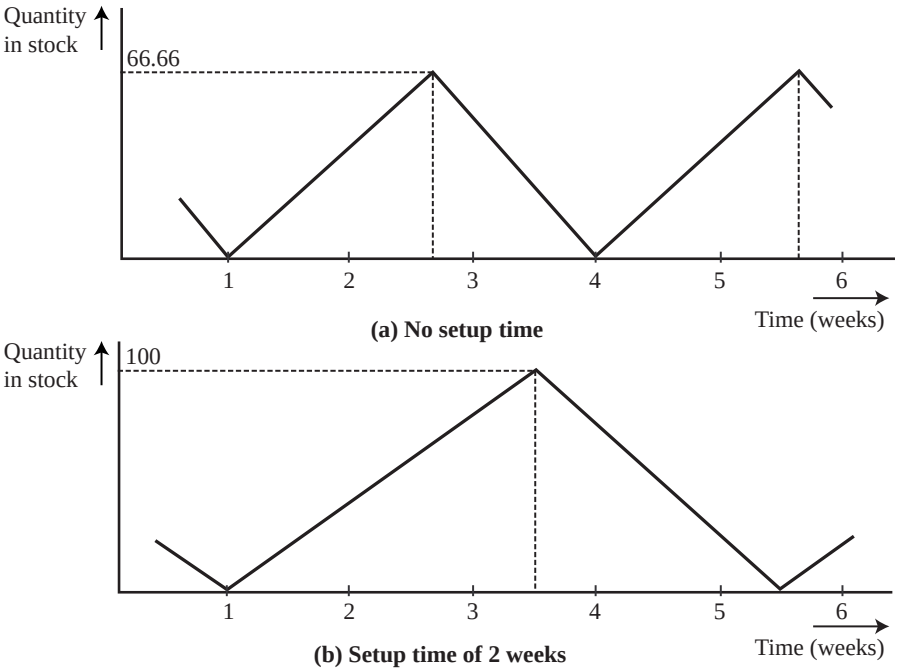

Fig. 7.1. Inventory levels in Example 7.2.1

If the setup time is longer than 9 days, then the cycle time has to be longer. For example, if a setup lasts 2 weeks (because of maintenance and cleaning), then the cycle time is 4.5 weeks. Figure 7.1.b depicts the inventory level over a cycle.

The next example highlights the differences between continuous and discrete settings.

Example 7.2.2 (Continuous Setting vs. Discrete Setting). Consider a production rate Q of 0.3333 items per day, a holding cost h of \$5.00 per item per day and a setup cost c of \$90.00. The demand rate D is 0.10 items per day. Applying the cycle length formula gives

$$x = \sqrt{\frac{60}{0.5(0.3333 - 0.1)}} = 22.678$$

and the number of items in a lot is $Dx = 2.2678$.

In a discrete setting such a number is not feasible. Consider the following discrete counterpart of this instance. The time to produce one item (or job) is $p = 1/Q = 3$ days. The demand rate is 1 item every 10 days. A lot of size k, k integer, has to be produced every $10k$ days. (The solution in the continuous

setting suggests that the optimal solution in the discrete setting is either a lot of size 2 every 20 days or a lot of size 3 every 30 days.) The total cost per day with a lot of size 1 every 10 days is $90/10 = \$9.00$. The total cost per day with a lot of size 2 every 20 days is

$$(90 + 7 \times 5)/20 = \$6.25$$

and the total cost per day with a lot of size 3 every 30 days is

$$(90 + 7 \times 5 + 14 \times 5)/30 = \$6.50.$$

So in a discrete setting it is optimal to produce every 20 days a lot of size 2.

7.3 Different Types of Items - Rotation Schedules

Consider again a single machine, but now with n different items. The demand rate for item j is D_j and the machine is capable of producing item j at a rate Q_j. In order to start a production run for item j, a setup cost c_j is incurred. We assume, for the time being, that this setup cost is sequence independent. In this section we determine the best production cycle that contains a single run of each item. Thus, the cycle lengths of the n items have to be identical. Such a schedule is referred to as a *rotation* schedule. The length of the cycle determines the length of each of the production runs. Hence there is only a single decision variable, the cycle length x. In order to determine the optimal cycle length, it is again necessary to find an expression for the total cost per unit time as a function of the cycle length x.

If setups require machine time, then it may not be possible to make the cycle length arbitrarily small since frequent setups may take up too much machine time.

The length of the production run of item j in a cycle is $D_j x/Q_j$. Assume that the inventory level at the beginning of the production run of item j is zero. During the production run, the level increases at rate $Q_j - D_j$ until it reaches level $(Q_j - D_j)D_j x/Q_j$. During the idle period, the inventory decreases at a rate D_j until it reaches zero and the next production run starts. So the average inventory level of item j is

$$\frac{1}{2}\Big(D_j x - \frac{D_j^2 x}{Q_j}\Big).$$

The facility incurs a setup cost c_j for each production run of item j. The average cost per unit time due to setups for item j is therefore c_j/x. The total average cost per unit time due to inventory holding costs and setup costs is therefore

$$\sum_{j=1}^{n} \Big(\frac{1}{2}h_j\Big(D_j x - \frac{D_j^2 x}{Q_j}\Big) + \frac{c_j}{x}\Big).$$

To find the optimal cycle length we take the derivative with respect to x and set it equal to zero, obtaining

$$\sum_{j=1}^{n} \Big(\frac{1}{2} h_j D_j \Big(1 - \frac{D_j}{Q_j} \Big) \Big) - \frac{\sum_{j=1}^{n} c_j}{x^2} = 0,$$

Straightforward algebra yields the optimal cycle length

$$x = \sqrt{\Big(\sum_{j=1}^{n} \frac{h_j D_j (Q_j - D_j)}{2 Q_j} \Big)^{-1} \sum_{j=1}^{n} c_j}.$$

The machine idle time during a cycle can be computed in a manner similar to the single item case. This idle time is equal to

$$x \Big(1 - \sum_{j=1}^{n} \frac{D_j}{Q_j} \Big).$$

The ratio $\rho_j = D_j / Q_j$ can be regarded as the utilization factor of the machine due to item j.

Consider the limiting case where the production rates are arbitrarily fast, i.e., $Q_j = \infty$ for $j = 1, \ldots, n$. In this special case the optimal cycle length is

$$x = \sqrt{\Big(\sum_{j=1}^{n} \frac{h_j D_j}{2} \Big)^{-1} \sum_{j=1}^{n} c_j}.$$

Example 7.3.1 (Rotation Schedules without Setup Times). Consider four different items with the following production rates, demand rates, holding costs and setup costs.

items	1	2	3	4
D_j	50	50	60	60
Q_j	400	400	500	400
h_j	20	20	30	70
c_j	2000	2500	800	0

The optimal cycle length x is 1.24 months and the total idle time is $0.48x$ $= 0.595$ months. Figure 7.2 displays the optimal rotation schedule. The total average cost per unit time can be computed easily and is $2155 + 2559 + 1627 + 2213 = 8554$.

As the setup cost of item 4 is zero, it is clear that a rotation schedule does not make sense here. It makes more sense to spread the production of item 4 uniformly over the cycle to reduce inventory holding costs. In the next section we consider this example again and allow for more general schedules.

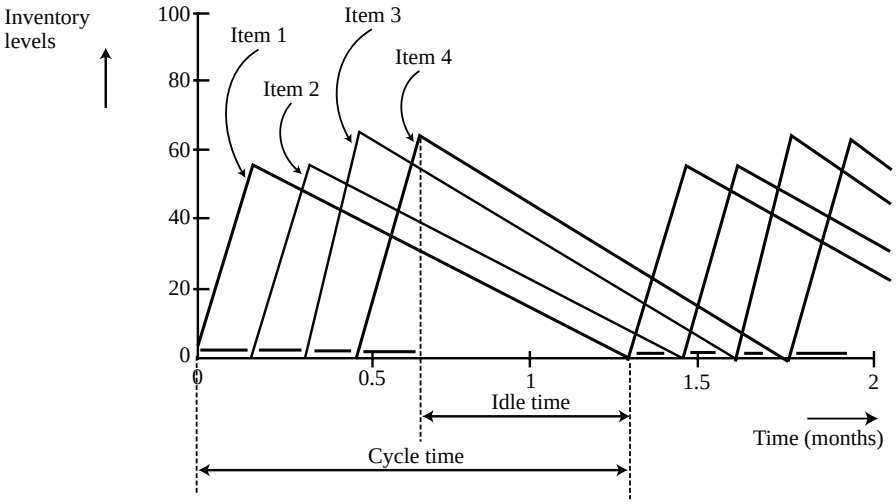

Fig. 7.2. Rotation schedule in Example 7.3.1

In the analysis above the order in which the different runs are sequenced does not matter. We assumed that there were no setup times and that setup costs were sequence independent. So, up to now, there was not any scheduling problem, only a lot sizing problem.

If there are setup times that are sequence independent, i.e., $s_{jk} = s_k$ for all j and k, then the problem still does not have a sequencing component, since the sum of the setup times does not depend on the sequence. If the sum of the setup times is less than the idle time in the rotation schedule computed above, the length of the rotation schedule remains optimal. If the sum of the setup times exceeds the idle time computed above, then the actual optimal cycle length has to be larger than the optimal cycle length obtained before. Actually, the optimal cycle length again turns out to be the cycle length that corresponds to a schedule in which the machine is never idle, i.e.,

$$x = \Big(\sum_{j=1}^{n} s_j\Big) \Big/ \Big(1 - \sum_{j=1}^{n} \rho_j\Big).$$

If the setup times are sequence dependent, then there is a sequencing problem and a sequence that minimizes the sum of the setup times has to be found. Minimizing the sum of the setup times in a rotation schedule is equivalent to the so-called *Travelling Salesman Problem (TSP)*, which can be described as follows. A salesman has to visit n cities and the distance from city j to city k is d_{jk}. His objective is to find a tour with the minimum total travel distance. That this TSP is equivalent to our sequencing problem can be shown easily. City j corresponds to item j and the distance from city j to

city k, d_{jk}, is equivalent to the setup time needed when item k follows item j, i.e., s_{jk}. The TSP is known to be NP-hard.

If, in the case of sequence dependent setup times, a sequence can be found that minimizes the sum of the setup times and this sum is less than the idle time in the rotation schedule, then the lot sizes computed above, as well as the sequence, are optimal.

However, if the optimal sequence results in a total setup time that is larger than the machine idle time obtained before, then the optimal cycle length has to be larger than the cycle length given in the formula above. The optimal cycle length then again will be such that the machine is always either producing or being setup for the next production run. In any case, the lot sizing problem and the scheduling problem can still be analyzed separately.

The scheduling problem with arbitrary setup times is known to be extremely hard. However, when the setup times have a special structure, an easy solution may exist. Consider, for example, the following setup times:

$$s_{jk} = 0, \qquad j \leq k$$

and

$$s_{jk} = (j-k)s, \qquad j > k.$$

An optimal sequence can be obtained by starting out with the item with the lowest index, continuing with the item with the second lowest index, and so on. At the end of the run of the item with the highest index, a changeover is made to the item with the lowest index in order to start a new cycle. This sequence is obtained by applying the *Shortest Setup Time first (SST)* rule, which is often used as a heuristic in cases with arbitrary setup times (see Appendix C).

Example 7.3.2 (Rotation Schedules with Setup Times). Consider the same four items as in Example 7.3.1. However, there are now sequence dependent setup times. There are $3! = 6$ possible sequences. The setup times are given in the table below.

k	1	2	3	4
s_{1k}	-	0.064	0.405	0.075
s_{2k}	0.448	-	0.319	0.529
s_{3k}	0.043	0.234	-	0.107
s_{4k}	0.145	0.148	0.255	-

This setup matrix is asymmetric, i.e., s_{jk} is not necessarily equal to s_{kj}.

Recall that in the case without setup times the cycle time is 1.24 months and the total idle time is 0.595 months. Since there are only six sequences, all sequences can be enumerated and the best one can be selected. The sequence $1, 4, 2, 3$ requires a total setup of 0.585 months, which is feasible and therefore optimal. However, if SST is used starting with item 1, then the sequence $1, 2, 3, 4$ is selected. This sequence requires a total setup of 0.635 months which exceeds the idle time under the optimal cycle.

7.4 Different Types of Items - Arbitrary Schedules

We now generalize the model described in the previous section to allow for schedules that are more general than rotation schedules. Within a cycle there may be multiple runs of any given item. For example, if there are three different items 1, 2 and 3, then the cycle $1, 2, 1, 3$ is allowed. There may be setup costs as well as setup times.

If ρ_j denotes the utilization factor D_j/Q_j of item j, then a feasible solution exists if and only if

$$\rho = \sum_{j=1}^{n} \rho_j < 1.$$

This necessary and sufficient condition is the same as the condition for the model in Section 7.3. It is intuitive that the setup times do not have any impact on this feasibility condition. The setup times do take up machine time; but, if a machine is operating close to capacity, then the cycle time and individual production runs just have to be made long enough in order to minimize the impact of the setup times.

In contrast to the model in the previous section for which there exists a closed form solution (at least, when the setup times are sequence independent), the problem considered in this section is very hard. There does not exist an efficient algorithm for this problem. However, there are good heuristics that usually lead to satisfactory solutions. In what follows, we describe one such heuristic for the case with sequence independent setup times, i.e., $s_{jk} = s_k$. Let $\mathcal{S}$ denote the set of all possible sequences of arbitrary length and j_l the index of the item produced in position l of the sequence. So $j_1, \ldots, j_\nu$, where $\nu \geq n$, denotes the production sequence of a given cycle. The sequence may contain repetitions. Consider the item that is produced in the l-th position of the sequence. If $j_l = k$, then item k is produced in the l-th position of the sequence. In the remainder of this section, the superscript l is used to refer to data related to the item produced in the l-th position of the sequence, e.g., $Q^l = Q_{j_l}$ and if the item in the l-th position is item k, then $Q^l = Q_{j_l} = Q_k$.

The production of the item in position l involves a setup cost c^l, a setup time s^l, a production time τ^l, and a subsequent idle time u^l which may be zero. If item k is produced in the l-th position, item k may be produced again within the same cycle. Let x denote the cycle time and v the time from the start of the production of item k in the l-th position till the start of the next production of item k (this may be in the same cycle or in the next cycle). So

$$v = \frac{Q^l \tau^l}{D^l}$$

and if $j_l = k$, then

$$v = \frac{Q_k \tau^l}{D_k}.$$

The highest inventory level is $(Q^l - D^l)\tau^l$. The total inventory cost for the production run of item k in position l is

$$\frac{1}{2}h^l(Q^l - D^l)\Big(\frac{Q^l}{D^l}\Big)(\tau^l)^2.$$

Let I_k denote the set of all positions in the sequence in which item k is produced and L_l all the positions in the sequence starting with position l (when item k is produced) up to, but not including, the position in the sequence where item k is produced next. The definition of L_l assumes that the sequence $j_1, \ldots, j_\nu$ repeats itself. Let $\mathcal{S}$ denote the set of all possible cyclic schedules. The ELSP can now be written as

$$\min_{\mathcal{S}} \min_{x,\tau^l,u^l} \frac{1}{x}\Big(\sum_{l=1}^{\nu}\frac{1}{2}h^l(Q^l - D^l)\Big(\frac{Q^l}{D^l}\Big)(\tau^l)^2 + \sum_{l=1}^{\nu} c^l\Big)$$

subject to

$$\sum_{j\in I_k} Q_k\tau^j = D_k x \qquad \text{for } k = 1, \ldots, n,$$

$$\sum_{j\in L_l}(\tau^j + s^j + u^j) = \Big(\frac{Q^l}{D^l}\Big)\tau^l \qquad \text{for } l = 1, \ldots, \nu,$$

$$\sum_{j=1}^{\nu}(\tau^j + s^j + u^j) = x$$

The first set of constraints ensures that enough time is allocated to the production of item k to meet its demand over the cycle. The second set ensures that enough of the item in position l is produced to meet the demand till the next time that item is produced.

The problem described above may be viewed as being composed of a master problem and a subproblem. The master problem focuses on the search for the best sequence $j_1, \ldots, j_\nu$ (an element of $\mathcal{S}$), and the subproblem must determine the optimal production times, idle times, and cycle length (τ^l, u^l, x) given the sequence.

That the subproblem is relatively simple can be argued as follows. If the sequence $j_1, \ldots, j_\nu$ is fixed, then the first set of constraints in the nonlinear programming formulation is redundant, since substitution of the third set into the first set yields

$$\sum_{j\in I_k}\Big(\frac{Q^j}{D^j}\Big)\tau^j = \sum_{j=1}^{\nu}(\tau^j + s^j + u^j),$$

which is the sum of the second set over all positions in I_k. So, given a fixed sequence, the nonlinear programming problem that determines the optimal production times and idle times can be formulated as follows:

$$\min_{x,\tau^l,u^l} \quad \frac{1}{x}\Big(\sum_{l=1}^{\nu}\frac{1}{2}h^l(Q^l-D^l)\Big(\frac{Q^l}{D^l}\Big)(\tau^l)^2 \;+\; \sum_{l=1}^{\nu}c^l\Big)$$

subject to

$$\sum_{j\in L_l}(\tau^j+s^j+u^j)=\Big(\frac{Q^l}{D^l}\Big)\tau^l \qquad \text{for } l=1,\ldots,\nu,$$

$$\sum_{j=1}^{\nu}(\tau^j+s^j+u^j)=x$$

The master problem, i.e., finding the best sequence $j_1,\ldots,j_\nu$, is more complicated. One particular heuristic yields good sequences in practice. This heuristic is in what follows referred to as the *Frequency Fixing and Sequencing (FFS)* heuristic. This FFS heuristic consists of three phases:

(i) The computation of relative frequencies phase.
(ii) The adjustment of relative frequencies phase.
(iii) The sequencing phase.

The first phase determines the relative frequencies with which the various items have to be produced. The number of times item k is produced during a cycle is denoted by y_k. In the second phase, these production frequencies are adjusted so they can be spaced out evenly over the cycle; the adjusted frequency of item k is denoted by y'_k. In the third and last phase these adjusted frequencies are used to produce an actual sequence.

The first phase determines, besides the relative frequencies y_k, also the corresponding run times τ_k. If the runs of item k are of equal length and evenly spaced, then the frequency y_k and the cycle time x determine the run time τ_k, i.e.,

$$\tau_k=\frac{\rho_k x}{y_k}.$$

To compute the y_k we relax the original nonlinear programming formulation by dropping the second set of the three sets of constraints. Without these interlinking constraints the actual sequence is no longer important. Optimizing over sequences now becomes optimizing over the cycle time x and run times τ_k or, equivalently, over production frequencies y_k.

Substitutions lead to the following modifications in the objective function of the original nonlinear programming formulation of the ELSP problem:

$$\frac{1}{x}\Big(\sum_{l=1}^{\nu}\frac{1}{2}h^l(Q^l-D^l)\Big(\frac{Q^l}{D^l}\Big)(\tau^l)^2+\sum_{l=1}^{\nu}c^l\Big)$$

$$=\frac{1}{x}\Big(\sum_{k=1}^{n}\frac{1}{2}y_k h_k(Q_k-D_k)\Big(\frac{Q_k}{D_k}\Big)(\tau_k)^2+\sum_{k=1}^{n}y_k c_k\Big)$$

$$
\begin{aligned}
&= \sum_{k=1}^{n} \frac{1}{2} h_k (Q_k - D_k) \Big(\frac{Q_k}{D_k} \Big) \rho_k \tau_k + \sum_{k=1}^{n} \frac{c_k \rho_k}{\tau_k} \\
&= \sum_{k=1}^{n} \frac{1}{2} h_k (Q_k - D_k) \tau_k + \sum_{k=1}^{n} \frac{c_k \rho_k}{\tau_k} \\
&= \sum_{k=1}^{n} \frac{a_k \tau_k}{\rho_k} + \sum_{k=1}^{n} \frac{c_k \rho_k}{\tau_k} \\
&= \sum_{k=1}^{n} \frac{a_k x}{y_k} + \sum_{k=1}^{n} \frac{c_k y_k}{x},
\end{aligned}
$$

where

$$
a_k \;=\; \frac{1}{2} h_k (Q_k - D_k) \rho_k \;=\; \frac{1}{2} h_k (1 - \rho_k) D_k.
$$

Of course, in any feasible schedule the relative frequencies y_k have to be integers. This implies that the run times τ_k cannot assume just any values, since they are determined by the relative frequencies. However, in order to make the problem easier, we delete the integrality constraints on the y_k and thus relax the constraints on the τ_k as well (basically deleting the first set of constraints in the original nonlinear programming formulation). Disregarding the integrality constraints on the y_k results in a relatively easy nonlinear programming problem.

$$
\min_{y_k, x} \sum_{k=1}^{n} \frac{a_k x}{y_k} + \sum_{k=1}^{n} \frac{c_k y_k}{x},
$$

subject to

$$
\sum_{k=1}^{n} \frac{s_k y_k}{x} \leq 1 - \rho.
$$

The constraint in this problem is equivalent to the last constraint in the original formulation. If the left hand side of this constraint is strictly smaller than the right hand side, then the sum of the setup times is less than the time the machine is not producing, implying there is still some idle time remaining.

Before presenting a solution for this simplified nonlinear programming problem some observations have to be made. First, a solution that is feasible for the simplified nonlinear programming problem may not be feasible for the original nonlinear programming problem; the solution to the simplified problem may require some tweaking. Second, it is clear that the simplified nonlinear programming problem has an infinite number of optimal solutions. If the solution $x^*, y_1^*, \ldots, y_n^*$, is optimal and

$$
\sum_{k=1}^{n} \frac{s_k y_k}{x} < 1 - \rho
$$

(i.e., the inequality is strict), then any solution $kx^*, ky_1^*, \ldots, ky_n^*$, with k integer, is also optimal. Basically, multiplying the first cyclic schedule by an integer k results in an identical cyclic schedule. A third observation is the following: if the inequality above is strict, then the solution $x^*, y_1^*, \ldots, y_n^*$ would also be optimal if all setup times are zero. However, if in the optimal solution

$$\sum_{k=1}^{n} \frac{s_k y_k}{x} = 1 - \rho,$$

then the setup times play an important role. The fact that there is no idle time implies that the optimal solution requires relatively high production frequencies with relatively short run times (possibly due to high holding costs or low setup costs). These high frequencies require that the maximum proportion of machine time, i.e., $(1 - \rho)$, is dedicated to setup times.

The nonlinear programming problem can be dealt with as follows. Incorporating the constraint in the objective function using a Lagrangean multiplier λ ($\lambda \geq 0$), results in an unconstrained nonlinear optimization problem with objective function

$$\min_{y_k, x} \sum_{k=1}^{n} \frac{a_k x}{y_k} + \sum_{k=1}^{n} \frac{c_k y_k}{x} + \lambda\Big(\sum_{k=1}^{n} \frac{s_k y_k}{x} - (1 - \rho)\Big).$$

Taking the partial derivative of this function with respect to y_k and setting it equal to zero yields

$$y_k = x\sqrt{\frac{a_k}{c_k + \lambda s_k}}.$$

The cycle length x can be adjusted so that the production frequencies take appropriate values (for example, one may choose the cycle length x so that the smallest frequency value is approximately equal to 1). If there are idle times, then λ is set equal to zero. If there are no idle times, then the λ has to satisfy the equation

$$\sum_{k=1}^{n} \Big(s_k\sqrt{\frac{a_k}{c_k + \lambda s_k}}\Big) = 1 - \rho,$$

since

$$\sum_{k=1}^{n} s_k y_k = (1 - \rho)x.$$

The solution y_k is unlikely to be integer. To find an integer solution that is close to the values obtained for y_k may require the construction of a long sequence with high frequencies.

The second phase of the FFS heuristic makes adjustments in the frequencies y_k. It has been shown in the literature that it is possible to find a new set of frequencies y_k' that are integers and powers of 2 with the cost of this

new solution being within 6% of the cost of the original solution. Of course, the run times of item k have to change then as well. The new run times, τ'_k, can be computed by assuming that the total idle time remains the same and the runs of item k are of equal length and equally spaced.

The third phase of the FFS heuristic generates the actual sequence. The heuristic used here has its roots in the heuristic used for scheduling n different jobs on a number of parallel machines to minimize the makespan. (Recall from the second section of Chapter 5 that the most popular heuristic for this problem is the LPT rule). Let

$$y'_{\max} = \max(y'_1, \ldots, y'_n).$$

For each item k, there are y'_k jobs with the same estimated processing time τ'_k (assuming that the lots will be equally spaced). Now consider a scheduling problem with $y'_{\max}$ machines in parallel and y'_k jobs of length τ'_k, $k = 1, \ldots, n$, (implying a total of $\sum_{k=1}^{n} y'_k$ jobs). There is an additional restriction in that item k with frequency y'_k must have the y'_k lots (jobs) placed on machines that are equally spaced. For example, if $y'_{\max} = 6$ and $y'_k = 3$, then there are two choices: the three jobs are assigned either to machines 1, 3 and 5 or to machines 2, 4 and 6. Now the following variation of the LPT heuristic can be used: the pairs (y'_k, τ'_k) are listed in decreasing order of y'_k. Pairs with identical frequencies y'_k are listed in decreasing order of the estimated processing time τ'_k. The pairs are taken from the list one by one starting at the top. When the pair (y'_k, τ'_k) is taken from the list, the corresponding y'_k jobs of length τ'_k are put on the machines (satisfying the spacing restriction) so that the maximum of the total processing assigned so far to the selected y'_k machines is minimized. After all pairs in the list have been assigned, the resulting sequences on the $y'_{\max}$ machines are concatenated, i.e., machine 1, followed by machine 2, and so on, to obtain a single sequence. This idea is based on the fact that after all jobs have been scheduled and the total processing is more or less equally partitioned over all the machines, the concatenated sequence will maintain the equal spacing of the various runs of any given item.

The following example illustrates the application of the FFS heuristic to an instance without setup times.

Example 7.4.1 (Application of the FFS Heuristic without Setup Times). Consider again the situation described in Example 7.3.1. However, now the schedule does not necessarily have to be a rotation schedule. There are setup costs but no setup times. Since item 4 has no setup cost and a fairly high holding cost, its production should be spread out as uniformly as possible in between the production of the other three items.

In order to find the frequencies y_k, we have to first solve the unconstrained optimization problem. Note that

$$(1 - \rho)x = 0.48x,$$

$$y_k = \frac{\rho_k x}{\tau_k},$$

and

$$a_k = \frac{1}{2} h_k (Q_k - D_k)\rho_k.$$

The following values can be computed easily.

items	1	2	3	4
D_j	50	50	60	60
Q_j	400	400	500	400
h_j	20	20	30	70
c_j	2000	2500	800	0
ρ_j	0.125	0.125	0.12	0.15
a_j	437.5	437.5	792	1785

It immediately follows that

$$\begin{aligned} y_1 &= 0.46x \\ y_2 &= 0.42x \\ y_3 &= 0.99x \\ y_4 &= \infty \end{aligned}$$

Suppose that the cycle time x is set equal to 2 months. This cycle time corresponds to the following approximate values for $y_1, \ldots, y_4$: $y_1 = y_2 = 1$, $y_3 = 2$ and $y_4 = 16$. The choice of y_4 is somewhat arbitrary but it has to be made high. The higher y_4, the more uniform the production of item 4 can be made in the final solution. Given a cycle time of 2 months and the production frequencies above, the runtimes τ_k of the four items are $\tau_1 = \tau_2 = 0.25$, $\tau_3 = 0.12$, $\tau_4 = 0.3/16$.

Now we apply the LPT-like heuristic. The number of machines in parallel is $y_{\max} = 16$. Item 4, the first to be assigned, is assigned to all 16 machines with all 16 processing times equal to 0.3/16. Item 3 is assigned next and is assigned to machines 1 and 9 with the two processing times equal to 0.12. Item 1 is then put on machine 5 and item 2 on machine 13. Concatenating the sequences of the 16 parallel machines results in the cyclic schedule

$$|\,4\,,3\,|\,4\,|\,4\,|\,4\,|\,4\,,1\,|\,4\,|\,4\,|\,4\,|\,4\,,3\,|\,4\,|\,4\,|\,4\,|\,4\,,2\,|\,4\,|\,4\,|\,4\,|.$$

Item 4 goes first for 0.3/16 months, followed by item 3 for 0.12 months. Item 4 goes next four times in a row, each time for 0.3/16 months (these four runs are separated in the final solution by idle times). Item 1 follows for 0.25 months. Item 4 goes again four times, and so on.

A feasibility check has to be done. It is clear that, in an ideal solution, the runs of each item are spaced evenly over the cycle. If the runs of an item are evenly spaced, we are assured that there are no stockouts. Attempting to

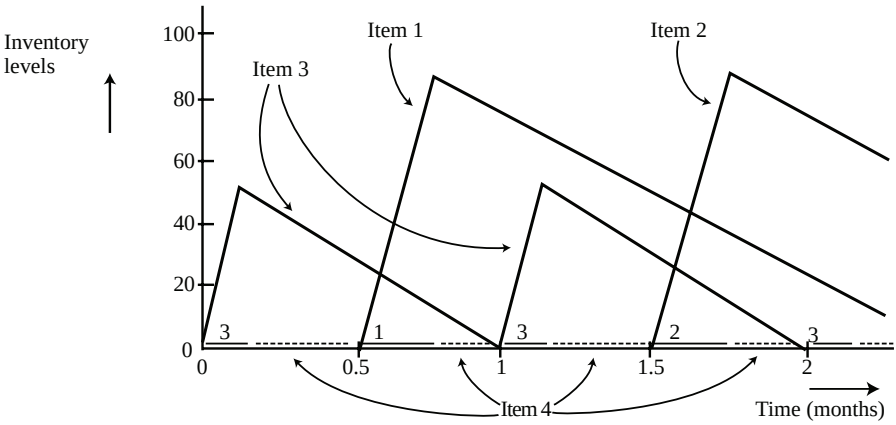

Fig. 7.3. Schedule in Example 7.4.1

uniformize the production of item 4 over the cycle results in many short runs that are either separated by idle times or by production runs of other items. We need to check whether, whenever items 1, 2, or 3 are produced, the stock of item 4 is sufficient to cover the demand in the periods that the other items are in production.

This solution could have been obtained in another way as well. As mentioned above, the y_4 was selected somewhat arbitrarily. The reason for choosing a high value is that it enables us to uniformize the item's production over the cycle. It is clear that the y_4 has to be chosen at least as large as the sum of all the other y's, i.e., at least 4. If $y_4 = 4$, then the algorithm yields the sequence

$$4, 3, 4, 1, 4, 3, 4, 2.$$

A schedule can now be constructed as follows. First the production runs of items 1, 2, and 3 are spaced evenly over the cycle and fixed. Then the production of item 4 is scheduled separately in between the remaining idle times. This production of item 4 is scheduled evenly over time (in *many* short runs). However, before any of the items 1, 2, or 3 has to go into production, a given amount of inventory of item 4 has to be built up. The inventory level of item 4 depends on the length of the runs of items 1, 2, and 3. It is only during these buildups of inventory that holding costs are incurred for item 4.

The entire schedule is depicted in Figure 7.3. The average total cost per unit time can be computed and is equal to

$$1875 + 2125 + 1592 + 190 = 5782.$$

Recall that the cost of the rotation schedule in Example 7.3.1 is 8554.

The next example illustrates the application of the FFS heuristic on an instance with setup times. In contrast to the previous example there is now, under the optimal sequence, no idle time on the machine.

Example 7.4.2 (Application of the FFS Heuristic with Setup Times). Consider the instance in the previous example but now with setup times. The setup times are sequence independent.

items	1	2	3	4
D_j	50	50	60	60
Q_j	400	400	500	400
h_j	20	20	30	70
c_j	2000	2500	800	0
s_j	0.5	0.2	0.1	0.2
ρ_j	0.125	0.125	0.12	0.15
a_j	437.5	437.5	792	1785

Note that, because of the nonzero setup time of item 4 the frequency of item 4 cannot be made arbitrarily high. In order to find the frequencies y_k, we first have to find a λ that satisfies the equation

$$\sum_{k=1}^{n} \left(s_k \sqrt{\frac{a_k}{c_k + \lambda s_k}} \right) = 1 - \rho.$$

It can be verified easily that $\lambda \approx 8000$ satisfies this equation. With this value of λ the y_k frequencies can be computed as a function of the cycle time x, i.e.,

$$y_k = x \sqrt{\frac{a_k}{c_k + \lambda s_k}}.$$

$$\begin{aligned} y_1 &= 0.27x \\ y_2 &= 0.33x \\ y_3 &= 0.70x \\ y_4 &= 1.05x \end{aligned}$$

If the cycle time x is fixed at 3 months, then the approximate values $y'_1, \ldots, y'_4$ can be either $(1, 1, 2, 2)$ or $(1, 1, 2, 4)$. Both solutions are power of two solutions. Compare these solutions with the frequency values in the previous example, i.e., $(1, 1, 2, 16)$. It is clear that, because of the setup times, the frequency of item 4 cannot be as high as in the previous example. There is simply not enough idle time for that many setups.

Consider the solution $x = 3$ with frequencies $(1, 1, 2, 2)$. The item sequence is $1, 3, 4, 2, 3, 4$. It has to be verified whether this solution is feasible. The idle time before taking setups into account is $0.48 \times 3 = 1.44$ months and the

total amount of setup time required is 1.3, which implies that the schedule is feasible. Actually, this means that the cycle time x can be made slightly smaller than 3, and a slightly smaller cycle time will give a better solution. (In the previous example, without setup times, the cycle length was 2 months; here the cycle time was made 3 months assuming that there would not be any idle time.) The average total cost per unit time can be computed as in the previous examples.

Consider the solution $x = 3$ with frequencies $(1, 1, 2, 4)$. The order of the items is

$$1, 4, 3, 4, 2, 4, 3, 4.$$

The total setup time required during a cycle is 1.7 months. This implies that a cycle length of 3 months is not feasible with these frequencies. In order to have these frequencies the cycle length has to be made larger (see Exercise 7.10).

7.5 More General ELSP Models

All models considered in the previous sections are single machine models. Some of these models can be extended fairly easily. For example, consider the model with multiple products on m identical machines in parallel. There are setup costs but no setup times. Any particular item has to be processed on one and only one of the m machines. The utilization factor of item j is again defined as $\rho_j = D_j/Q_j$ and in order for a feasible solution to exist we must have

$$\sum_{j=1}^{n} \rho_j \leq m.$$

Suppose that the schedule for each of the machines has to be a rotation schedule. If the cycle times of the m rotation schedules have to be equal, the problem is relatively easy and not much different from the one described in Section 7.3. The only additional issue that needs to be resolved is the assignment of the items to the different machines. Assuming that each item has to be produced on one and only one machine, the loads have to be balanced and the sum of the ρ_j's of the items assigned to any one machine has to be less than one. To find a good balance or, equivalently, a good partition of the n different items over the m machines, we can use the LPT heuristic with the ρ_j values playing the role of processing times. The LPT heuristic (used for minimizing the makespan in a parallel machine environment) will result in a reasonably good assignment of the items to the m machines.

Example 7.5.1 (Rotation Schedules with Machines in Parallel). Consider the situation in Example 7.3.1. Instead of a single machine, we now have 2 machines in parallel. The production rate of each of the two machines is half the production rate of the machine in Example 7.3.1. The data are presented below.

items	1	2	3	4
D_j	50	50	60	60
Q_j	200	200	250	200
h_j	20	20	30	70
c_j	2000	2500	800	0

Because the two machines have the same cycle length, the formula in Example 7.3.1 can be used here as well. The optimal cycle x in this case turns out to be 1.35 months (the optimal cycle length with a single machine at twice the speed is 1.24 months).

However, if the cycle times of the m rotation schedules are allowed to be different, then the problem becomes more difficult. We can reduce total cost by taking advantage of different cycle times. Again, an assignment of the items to the machines has to be found while maintaining a proper machine load balance. There is now an additional difficulty. If an item with a cost structure that favors a long cycle time is assigned to the same machine as an item with a cost structure that favors a short cycle time, then the solution is not likely to be a good one. One can deal with this difficulty as follows. Consider each item as a single product model (as in Section 7.2) and compute its cycle time. Rank the items in decreasing order of their cycle times. Start taking the items from the top of the list and put them on one machine. Keep assigning items to this machine until its capacity is exhausted, i.e., the allocation of an item makes the sum of the ρ_j's larger than one. This last item is then reallocated to the second machine, and so on. This procedure may not lead to a good load balance, and may even lead to an infeasible solution (i.e., the sum of the ρ_j's on the last machine may be larger than one). If that is the case, then items on adjacent machines have to be swapped in order to obtain a better balance.

The parallel machine model with rotation schedules and sequence dependent setups is of course harder. The assignment of items to the m machines now has to consider machine balance, preferred cycle times, as well as setup times on all machines. The setup time structure becomes especially important when there are large differences between setup times. Very little research has been done on this problem.

When the schedules on the different machines do not have to be rotation schedules, i.e., the parallel machines generalization of the model considered in Section 7.4, the problem is even harder. However, now it is no longer necessary to assign items with similar cycle times to the same machine. It is clear that in the parallel machine environment there are still many unresolved issues that require more research.

Another important extension of the single machine setting is the environment with machines in series, i.e., the flow shop. Consider a single machine feeding another single machine with the production rates and setup costs of the two machines being identical. Hence the cost structures of an item on the upstream machine and on the downstream machine are the same. The ma-

chines can be scheduled and synchronized so that the items, when they leave the upstream machine, can immediately start their processing on the downstream machine without having to wait. Because of this synchronization, the results in Sections 7.3 and 7.4 can be extended to the case of similar machines with identical costs in series.

Example 7.5.2 (Rotation Schedules with Machines in Series). Consider the same product mix as in Example 7.3.1. However, now instead of a single machine, we have two machines in series. After an item has completed its processing on the upstream machine, it has to go to the downstream machine and complete its processing there. There are setup costs but no setup times. The setup costs of an item on the two machines are the same. A rotation schedule is needed for the two machines (i.e., the same rotation schedule must be used for both machines).

If the rotation schedule is such that an item with a long processing time (i.e., with a low production rate) is followed by an item with a short processing time (i.e., with a high production rate), then the item with the short processing time may have to wait between the two machines. Assume that at the beginning of the rotation schedule machine 1 starts with the production of the item with the shortest processing time (i.e., with the highest production rate), it then continues, without stopping, with the item with the second shortest processing time, and so on. After it completes the run of the item with the longest processing time machine 1 remains idle until it is necessary to start the new cycle. In this way any item that comes out of machine 1 can start immediately on machine 2 without having to wait, i.e., there is no Work-In-Process in between the two machines.

The inventory costs of the finished goods are exactly the same as in the single machine case, so this system can be analyzed as a single machine. However, there are now two setup costs, instead of only one. This implies that the optimal cycle length is $\sqrt{2} = 1.4142$ times longer than the optimal cycle length for a single machine. This result can be extended easily to m identical machines in series.

The results in the previous example can be further generalized. Consider m machines in series with identical production rates for each product type, but different setup costs. This problem can still be reduced to a single machine problem with production rates identical to those of one of the machines in the original problem. However, now the setup costs have to be set equal to the sum of the setup costs of the original m machines, i.e., the setup cost for item j in the new problem is

$$c_j = \sum_{i=1}^{m} c_{ij}.$$

When the machines do not have identical production rates for each product type the problem is not that easy. Consider first the case with two machines that have identical setup costs but different speeds. But the speed structure

is uniform over the items, i.e., the production rate of item j on machine i is $Q_{ij} = v_i Q_j$, where v_i is a speed factor of machine i. One approach is to first analyze the slow machine as a single machine in isolation and then adapt the fast machine accordingly (since the high speed machine provides more flexibility). The schedule of the fast machine can be adapted in such a way that there is no WIP in between the two machines. This model can then be analyzed as a single machine with the production rates of the slow machine and setup costs that are the sum of the setup costs of the two machines.

When the production rates are not uniform, it may become necessary to schedule Work-In-Process (WIP) between the machines. The carrying cost of the WIP in between the machines may be different from the holding cost of the finished goods. This makes the model more complicated. Very little research has been done on this problem.

An even more general machine environment is the flexible flow shop, i.e., a number of stages in series with at each stage a number of machines in parallel. Under very special conditions optimal rotation schedules can be determined for such a machine environment. For example, consider two stages in series with at each stage two machines in parallel. For any given product type the production rates of the four machines are the same and so are the setup costs. Under these circumstances there is no need for any WIP in between two stages. This makes it possible to determine the optimal rotation schedules relatively easily when the cycle times of all four machines have to be the same.

7.6 Multiproduct Planning and Scheduling at Owens-Corning Fiberglas

Owens-Corning Fiberglas is a leading manufacturer of fiberglass products and has a large manufacturing facility in Anderson, South Carolina. In its manufacturing process, molten fiberglass is formed and the glass is spun onto spools of various sizes. This material is used to weave fabric and to produce chopped strand mat. Fiberglass mat is sold in rolls of various widths and weights, treated with one of three process binders, and trimmed at one or both edges or not at all. The demand for the products comes mainly from the marine industry for the manufacture of boat hulls, and from the residential construction industry for bath tubs and shower booths.

At the time when a production planning and scheduling system was developed for this facility, the product line consisted of over 200 distinct mat items. Twenty-eight of these represented over 80% of the total annual demand and were treated as high volume standard (stock) products. The remaining items were made to order. The manufacturing facility had two main production lines referred to as Mat Lines 1 and 2. Line 1 had approximately three times the capacity of Line 2 and could produce mat 76 inches wide, whereas Line 2 was limited to 60 inches. The product came off these lines in the form of 175- to 230-pound cylinders. The average cost of down time on Line 1 was

approximately $300 per hour; the average cost of down time on Line 2 was less. Maintenance costs were related to the frequency of job changeovers. In addition, each time a product change was made on a line, there was a sequence dependent setup cost partly due to material waste. The monthly costs due to setups ranged from $15,000 to $50,000 (with approximately 75 job changes and 50 hours of down time).

The production planning and scheduling system developed for the mat lines focused on three issues, namely aggregate planning (focusing on inventory costs and workforce scheduling), production run quantities and lot sizing (taking line assignments and inventory levels into account), and detailed line sequencing of Make-to-Stock and Make-to-Order products (taking setup costs into account). The system developed for the mat lines consisted therefore of three main modules, namely,

(i) the aggregate planning module,
(ii) the lot sizing module, and
(iii) the sequencing module.

The aggregate planning module used as input the aggregate demand forecast for the next twelve months. Its objective was to minimize the sum of direct payroll costs, overtime costs and hiring and firing costs. The time horizon ranged from three to twelve months. The optimization method in this module was based on a production switching heuristic; this rule considered inventory levels and forecasts of future demand and, based on these data, determined the appropriate production rates. The output of this model included targets with respect to aggregate inventory levels, production rates and levels of employment.

The output of the aggregate planning module served as input to the lot sizing module. The lot sizing module also required a detailed short term demand forecast for each stock item. The time horizon considered in this module was up to three months. The output from this module were the line assignments and the lot sizes. The optimization in this module was based on a linear program formulation. The objective was to minimize all the relevant setup costs and production costs subject to several sets of constraints. The inventory level constraints ensured that aggregate inventory levels and safety stock levels were met and the production balance constraints guaranteed demand satisfaction as well as inventory conservation. The linear program was solved using the MPSX package and the program was run on a monthly basis providing the plant with specific inventory levels, lot sizes and line assignments for the coming months.

The output of the lot sizing module served as input for the sequencing module. The time horizon of this module was one month, and its main objective was the minimization of the sequence dependent setup costs. The dominant components of setup costs were direct downtime and mat waste. Changeovers were classified as fiber changes, width changes, weight changes, and slitter changes. A distinction could be made within each family of changeovers based upon the direction of the change. For example, it was easier to decrease weight and width than to increase them. The sequencing heuristic was based on sim-

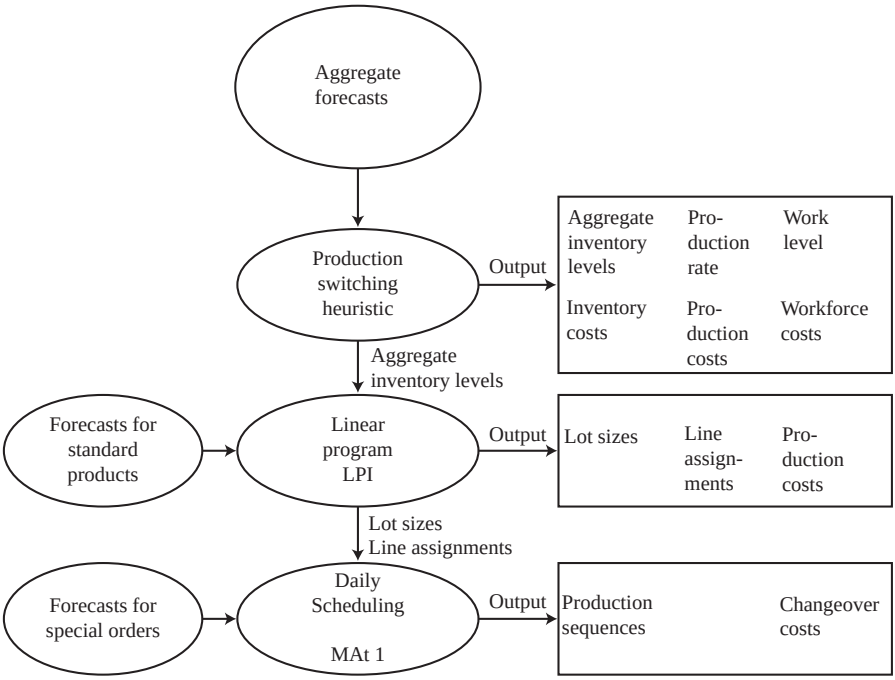

Fig. 7.4. Overview of system at Owens-Corning Fiberglass

ple dispatching rules and the sequencing module was run on a weekly basis. The entire system is depicted in Figure 7.4.

Implementation of the system led to major improvements in the operation of the plant. The average number of changeovers went down from an average of 70 before the system's implementation to an average of 40 after the system's implementation.

The Owens-Corning Fiberglas environment is somewhat similar to the setting described in Section 7.4. However, Owens-Corning had two machines in parallel instead of the single machine in Section 7.4 (a parallel machine environment was considered in Section 7.5). Note that the FFS heuristic described in Section 7.4 is based on a nonlinear programming formulation, whereas the lot sizing module in the Owens-Corning Fiberglas system was based on a linear programming formulation.

7.7 Discussion

The models discussed in this chapter have similarities as well as differences with the models for the the flexible assembly systems described in Chapter 6. In both chapters the planning horizons are basically unbounded. (This is in

contrast to the models described in Chapters 4 and 5, which all have a finite number of jobs.) However, the objectives considered in Chapter 6 are fundamentally different from the objectives considered in this chapter. In Chapter 6, the typical objective is to maximize the throughput or, equivalently, to minimize the cycle time. In this chapter, the throughput is basically given, since the demand levels are known. The objective is to minimize the sum of the inventory carrying costs and the setup costs. Nonetheless, the objectives in this chapter display some similarities with the objectives in paced assembly systems.

The models considered in this chapter are very important for industries that produce Make-To-Stock and for environments with setup times and costs. Examples of these industries include the paper industry, the aluminum industry and the steel industry. If one compares the models described in this chapter with the problems that have to be solved in those industries, then a number of issues arise. The problems in practice are, of course, more complicated than the models considered in this chapter. Often, if there are multiple machines in parallel, the production rates of any given item on the various machines may vary. Run lengths have in practice, besides an impact on the inventory costs and the total change-over costs, also an effect on various other factors, including

(i) the quality of the finished product,
(ii) the production yield or the amount of waste incurred,
(iii) the productivity of the facility and its total production capacity.

These three factors, which are not independent, are seldom included in medium term or long term planning models. The three factors are somewhat related to one another. The quality of the products in process industries (which typically is a continuous measure rather than a discrete measure) depends strongly on the length of the run. The longer the length of the run, the higher the average quality of production. In the process industries there is usually also a yield or waste problem (cutting stock or trim related issues). If the run length is very small, then the average waste tends to increase. So the more changeovers there are, the lower the average quality of the product, and the lower the productivity and the capacity. The cost of machine capacity is basically determined by opportunity costs (i.e., the shadow prices or dual prices of the resources involved).

Practical problems are often a combination of Make-To-Stock and Make-To-Order. The Make-To-Stock aspects involve problems such as those described in this chapter whereas the Make-To-Order aspects are related to job shop scheduling problems. Researchers have been analyzing inventory problems in which the facilities are assumed to be set up in series. This area of research, often referred to as multi-echelon inventory theory or supply chain management, is considered in the next chapter.

Exercises

7.1. Consider four different products with the following demand rates, production rates, holding costs and setup costs.

items	1	2	3	4
D_j	50	50	60	60
Q_j	400	500	500	400
h_j	20	20	30	70
c_j	2000	1000	1000	100

(a) Find the optimal rotation schedule. Determine its cycle length, and the total idle time.

(b) Suppose now that item 4 can be produced many times during a cycle. Items 1, 2 and 3 still can be produced only once during a cycle. Find the optimal production schedule. How does the optimal cycle length compare with the original optimal cycle length?

7.2. Consider two identical machines in parallel. Four items have to be produced.

items	1	2	3	4
D_j	50	50	60	60
Q_j	200	200	300	300
h_j	20	20	30	70
c_j	2000	2500	800	0

(a) Find the optimal rotation schedule assuming that the cycle lengths of the two machines have to be the same. Compute the total average cost per unit time.

(b) Find the optimal rotation schedules of the two machines assuming the cycle lengths of the two machines do not have to be the same (determine which items have to be combined with one another on the same machine to obtain the best result). Compute the average cost per unit time and compare the result with the result found under (a).

7.3. Consider the following two stage production process in a paper mill with a downstream converting operation. At the first stage there is a single paper machine. The output of this operation consists of large rolls of paper. The second stage is a single machine cutting operation that produces cutsize paper. To simplify the problem assume that only two items have to be produced. Also, each item that comes out of the second stage corresponds to one of the items that comes out of the first stage. The production rates, setup costs and holding costs are different at the two stages. In the table below Q_{ij} denotes the production rate of item j at stage i, c_{ij} the setup cost of item j at stage i, and h_{ij} the holding cost of item j after processing at stage i (so h_{1j} denotes

the holding cost of keeping item j in inventory in between the two stages, while h_{2j} denotes the holding cost of item j as a finished good).

items	1	2
D_j	100	50
Q_{1j}	400	400
Q_{2j}	600	1000
h_{1j}	20	20
h_{2j}	60	80
c_{1j}	3000	2500
c_{2j}	1000	1250

The schedules at both stages have to be rotation schedules (i.e., it is, for example, not allowed to produce item 1 at the first stage for a while, leave the machine idle for some time, produce item 1 again, and then item 2).

(a) Assuming that the cycle length x of the two stages have to be the same, what is the cycle length with the minimum total cost?

(b) Assume that the cycle lengths at the two stages are allowed to be different. Determine the optimal cycle lengths of the two stages.

7.4. Connsider the environment with two machines in parallel in Example 7.5.1. Suppose now that the production rate of one machine is .7 times the production rate of the machine in Example 7.3.1 and the production rate of the second machine .3 times.

a) Determine the optimal rotation schedules when both machines must have the same cycle time.

b) Determine the optimal rotation schedules when the two machines do not have to have the same cycle times.

7.5. Consider the data in Example 7.3.1. Consider now m identical machines in parallel. The production rate of item j on any one of the machines is Q_j/m. (This implies that the total production capacity does not depend on the number of machines in parallel.) Assume that all the machines have to be scheduled according to rotation schedules with the same cycle time x. Compute the optimal x and the total cost for $m = 2, 3, 4$. Plot the total cost against m.

7.6. Consider m identical facilities with each facility having a production rate Q_j/m. There are no setup times. Assume that all the facilities have to follow rotation schedules with the same cycle length x. Derive an expression for the optimal cycle length x. How does x depend on m? Discuss the monotonicity and the convexity of the function.

7.7. Consider a paper mill with two paper machines. There are 5 different types of paper that have to be produced. Items 1 and 2 have to be produced on machine 1 and item 3 has to be produced on machine 2. Items 4 and 5 can be produced on either one of the two machines.

items	1	2	3	4	5
D_j	60	60	80	80	100
Q_j	200	200	300	300	400
h_j	20	30	40	20	20
c_j	3000	2000	800	4000	1500

(a) Determine the optimal rotation schedule assuming that the cycle lengths have to be the same.

(b) Determine the optimal rotation schedules assuming the cycle lengths do not have to be the same.

(c) Determine the optimal schedule if the schedule on machine 1 has to be a rotation schedule and the schedule on machine 2 may be an arbitrary schedule.

7.8. Consider the following generalization of the paper making facility of Exercise 7.3. Again, there are two stages. The entire facility produces three different items. However, the paper machine at the first stage produces only two different intermediate products. One of the intermediate products that comes out of stage 1 is used at stage 2 to produce items 1 and 2. (This implies that the data correponding to items 1 and 2 regarding stage 1 are the same.) The other intermediate product that comes out of stage 1 is converted at stage 2 into item 3.

items	1	2	3
D_j	50	70	100
Q_{1j}	250	250	300
Q_{2j}	200	400	300
h_{1j}	30	30	40
h_{2j}	40	20	30
c_{1j}	2000	2000	900
c_{2j}	1000	3000	2000

Determine the optimal rotation schedule and its cycle length.

7.9. Consider the setting in Exercise 7.2. Instead of a single stage with two machines in parallel, we have now two stages in series with two machines in parallel at each stage. All four machines are identical with regard to production rates and setup costs (the data being the same as in Exercise 7.2). Determine the optimal rotation schedules of the four machines assuming that the cycle times of the four machines are the same.

7.10. Consider the setting in Example 7.4.2.

(a) What is the minimum cycle time when the frequency values are $y'_1 = y'_2 = 1$ and $y'_3 = y'_4 = 2$? Compute the total average cost of this solution.

(b) What is the minimum cycle time when the frequency values are $y'_1 = y'_2 = 1$, $y'_3 = 2$ and $y'_4 = 4$? Compute the total average cost of this solution.

(c) Compare the results obtained under (a) and (b) with the total average cost obtained in Example 7.4.1. Explain your results.

Comments and References

Various books and monographs focus on lot sizing and scheduling; see, for example, Haase (1994), Brüggemann (1995), Kimms (1997), and Zipkin (2000). Several excellent survey papers cover this topic also in depth, see Graves (1981) and Drexl and Kimms (1997).

The material in Section 2 is very basic. The EOQ formula was first mentioned by Harris (1915) and studied in detail by Wilson (1934). This material is covered in every elementary textbook on production planning and operations management.

Maxwell (1964) did an exhaustive study of rotation schedules. Gallego (1988) and Gallego and Roundy (1992) generalized these results allowing for backorder costs. Jones and Inman (1989, 1996) made an in depth study of the worst case behavior of rotation schedules and compared rotation schedules with other types of schedules. Gallego and Queyranne (1995) extended some of these results.

The FFS heuristic, described in Section 7.4, for generating arbitrary schedules is due to Dobson (1987, 1992). Gallego and Shaw (1997) established the NP-hardness of the ELSP with arbitrary cyclic schedules.

A fair amount of work has been done on lot scheduling in more complicated machine environments; see Crowston, Wagner and Williams (1973), Carreño (1990), Brüggemann (1995), Jones and Inman (1996), Chao and Pinedo (1996), and Pinedo and Chao (1999).

The production planning and scheduling system developed for Owens-Corning Fiberglas is discussed in Oliff and Burch (1985).